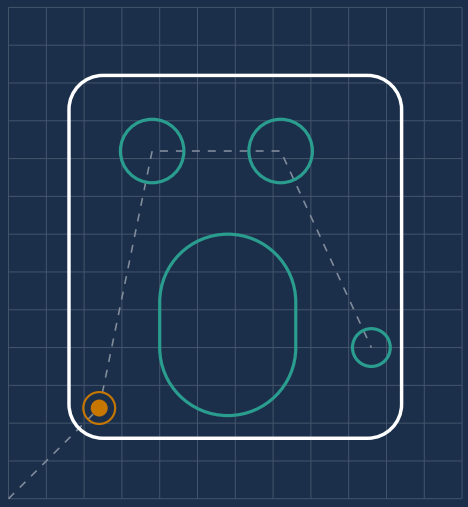
HP-GL PEN PLOTTER FOR THE TEKNIC CLEARCORE

# ClearHPGL

## User's Manual

Firmware, desktop app, and machine setup.
Plot from AutoCAD and other CAD software
over USB, serial, or Ethernet.



Firmware version 0.2.0 | CH-UM-100, Revision C


AUTHOR AND DEVELOPER

**Adam G. Sweeney**

agsweeney@gmail.com



Written in ASD-STE100 Simplified Technical English.
Edited to *The Chicago Manual of Style*, 18th edition.

November 2026


MIT License

**ClearHPGL User's Manual**
Document CH-UM-100, Revision C, November 2026
For ClearHPGL firmware version 0.2.0 and the ClearHPGL desktop app

Copyright © 2026 Adam G. Sweeney. All rights reserved.

The ClearHPGL software is released under the MIT License. The full license text is in the file `LICENSE` at the root of the source repository. The software builds on code by others that keeps its own license; the file `THIRD_PARTY_NOTICES.md` lists it. The software is supplied "as is," without warranty of any kind.

ClearCore™ and ClearPath™ are trademarks of Teknic, Inc. HP-GL and HP-GL/2 are trademarks of HP Inc. AutoCAD is a trademark of Autodesk, Inc. Windows is a trademark of Microsoft Corporation. ASD-STE100® is a trademark of ASD (AeroSpace, Security and Defence Industries Association of Europe). All other product names are trademarks of their respective owners. Their use in this manual identifies products and does not imply endorsement.

**Status.** ClearHPGL is experimental. It operates on one bench machine (a ClearCore, ClearPath-SD servos on X and Y, and a pen on an output) and plots real jobs, also directly from AutoCAD. It is not certified for production or for applications where safety is critical. Read chapter 2 before you operate the machine.

**Editorial conventions.** This manual uses ASD-STE100 Simplified Technical English for its text and *The Chicago Manual of Style*, 18th edition, for its apparatus: capitalization of titles, punctuation, numbers, captions, notes, and bibliography. The preface gives the details.

| Revision | Change |
|---|---|
| A, November 2026 | First issue, for firmware version 0.1.0. |
| B, November 2026 | CoreXY and H-bot machines: settings `kinematics` (now the first setting), `pulley_dia_mm`, `swap_xy`, and `motor_max_mm_s`. `origin` restarts the motor step counters. Firmware update from the app, and the ClearPath scope. |
| C, November 2026 | For firmware version 0.2.0. Limit and home switches, homing on the switches or with ClearPath hard-stop homing, and homing before each job (chapter 6). Console commands `home`, `jog`, `swinfo`, and `sim`, and new `status` fields. `abort` stops along the path with a ramp. In the app: the limits and homing wizard, the Home button, and the homed and limit state in the status bar. |

# Contents

**Figures** vii

**Tables** vii

**Preface** xi

**1 Introduction** 1
- 1.1 What ClearHPGL Is . . . 1
- 1.2 Features . . . 1
- 1.3 How a Job Goes Through the Board . . . 1
- 1.4 What ClearHPGL Does Not Do . . . 2

**2 Safety** 5
- 2.1 Status of the Software . . . 5
- 2.2 Motion That You Do Not Expect . . . 5
- 2.3 Faults and the Emergency Stop . . . 5
- 2.4 Lasers . . . 6
- 2.5 Electrical Safety . . . 6

**3 Hardware and Connections** 7
- 3.1 Parts . . . 7
- 3.2 Connector Map . . . 7
- 3.3 ClearPath Servos . . . 7
- 3.4 CoreXY and H-Bot Machines . . . 9
- 3.5 Pen Lift . . . 9
- 3.6 Emergency Stop . . . 10
- 3.7 Links to the Computer . . . 10

**4 Install the Firmware** 11
- 4.1 What You Need . . . 11
- 4.2 Update from the App . . . 11
- 4.3 Build and Load the Firmware . . . 12
- 4.4 Make Sure That the Firmware Operates . . . 13
- 4.5 Settings of Other Firmware . . . 13

**5 First Setup** 15
- 5.1 Open the Console . . . 15
- 5.2 Set the Kinematics . . . 15
- 5.3 Set the Mechanics . . . 16
- 5.4 Set the Plot Area and the Origin . . . 17
- 5.5 Make Sure That the Axes Move Correctly . . . 17
- 5.6 Set the Pen Lift . . . 19
- 5.7 Set the Speeds . . . 19

5.8 Set the Network . . . 19
5.9 Do a Test Plot . . . 20

**6 Limits and Homing** **21**
6.1 How the Board Uses the Switches . . . 21
6.2 Connect the Switches . . . 22
6.3 Set Up the Switches With the Wizard . . . 22
6.4 Set Up the Switches in the Console . . . 23
6.5 Homing . . . 24
6.6 Homing Errors . . . 24
6.7 Test Without Switches . . . 25

**7 The ClearHPGL Desktop App** **27**
7.1 Install the App . . . 27
7.2 The Main Window . . . 27
7.3 Connect to the Plotter . . . 28
7.4 Open a Drawing . . . 29
7.5 The 3D View . . . 29
7.6 Plot . . . 30
7.7 Move the Pen . . . 31
7.8 Plotter Settings . . . 31
7.9 ClearPath Scope . . . 33
7.10 Limits and Homing . . . 34
7.11 Other Functions . . . 34

**8 Plot from CAD Software** **35**
8.1 AutoCAD . . . 35
8.2 Other Software . . . 36
8.3 Send a File without a Driver . . . 36

**9 How the Board Processes a Job** **37**
9.1 One Source at a Time . . . 37
9.2 The Job Optimizer . . . 37
9.3 Arcs and Circles . . . 38
9.4 Speed . . . 38
9.5 Coordinates and Clipping . . . 39
9.6 Pen Changes . . . 39
9.7 Abort a Job . . . 39

**10 Troubleshooting** **41**
10.1 Read the State of the Board . . . 41
10.2 Faults . . . 41
10.3 Diagnostics . . . 42
10.4 Problems and Remedies . . . 42

**A Console Commands** **45**

**B Settings** **49**
B.1 Machine . . . 49
B.2 Motion . . . 49
B.3 Pen . . . 50
B.4 Safety and Jobs . . . 50
B.5 Limits and Homing . . . 51
B.6 Serial Port and Network . . . 52

**C HP-GL Support** **53**
C.1 Syntax . . . 53
C.2 Instructions . . . 53
C.3 Errors and Status . . . 54
C.4 Device Control (ESC .) . . . 54
C.5 Flow Control . . . 55

**D Network and Command-Line Client** **57**
D.1 Ports . . . 57
D.2 Discovery . . . 57
D.3 The Command-Line Client . . . 57
D.4 Test without a Board . . . 58

**Glossary** **59**

**Bibliography** **61**

# List of Figures

1.1 The path of a job through ClearHPGL . . . . . 2

3.1 Connections of a ClearHPGL plotter. Dashed parts are optional. . . . . 8

4.1 Update board firmware, with the board read on COM3 . . . . . 12

7.1 The main window of the ClearHPGL app, before a connection . . . . . 28
7.2 The Plotter settings dialog, with the factory settings . . . . . 32
7.3 The ClearPath scope, on its simulated motor . . . . . 33

# List of Tables


1 Where to start . . . xi
2 Typographic conventions . . . xiii

1.1 Functions not in firmware version 0.2.0 . . . 3

3.1 Parts of a ClearHPGL plotter . . . 7
3.2 ClearCore connectors used by ClearHPGL . . . 8
3.3 Values of `estop_di6` . . . 10

5.1 When a setting can change . . . 16
5.2 Corrections for a CoreXY machine after `goto 20 0` . . . 18
5.3 Motion settings . . . 19

6.1 What a limit switch does . . . 22
6.2 Homing errors . . . 25

7.1 Controls of the 3D view . . . 30

8.1 AutoCAD drivers for ClearHPGL . . . 35

9.1 Values of `optimize` . . . 37

10.1 Fields of `status` . . . 41
10.2 Faults . . . 42
10.3 Problems and remedies . . . 43

A.1 Console commands . . . 45
A.2 Common console errors . . . 47

B.1 Machine settings . . . 49
B.2 Motion settings . . . 50
B.3 Pen settings . . . 50
B.4 Safety and job settings . . . 51
B.5 Limit and homing settings . . . 51
B.6 Serial port and network settings . . . 52

C.1 HP-GL instructions . . . 53
C.2 Error codes (`OE`) . . . 54
C.3 Status bits (`OS`) . . . 54
C.4 Device-control sequences . . . 54
C.5 Flow control on each link . . . 55

D.1 Network ports of the board . . . 57
D.2 Commands of the client . . . 57

# Preface

## Purpose of This Manual

This manual tells you how to build, set up, and operate a pen plotter that uses ClearHPGL. It tells you how to install the firmware on a Teknic ClearCore, how to connect the servos, the pen lift, and the emergency stop, and how to set the board for your machine. It also tells you how to plot from the ClearHPGL desktop app, from AutoCAD, and from other software.

ClearHPGL is firmware for the ClearCore. With it, the ClearCore and two ClearPath servos operate as an HP-GL pen plotter. A CAD program sends a drawing through a standard plotter driver, and the machine draws it. The optional desktop app shows a preview of the drawing, sends jobs, and shows the pen as it moves.

## Audience

This manual is for persons who build and operate the machine. You must know how to connect motors and switches to an industrial controller, and how to use a serial terminal and a Windows computer. Knowledge of HP-GL helps, but it is not necessary. The glossary defines the terms that this manual uses.

## How to Use This Manual

Table 1 shows where to start for each task.

**Table 1.** Where to start

| Task | Read |
|---|---|
| Learn what ClearHPGL does and does not do | Chapter 1 |
| Operate the machine safely | Chapter 2 |
| Connect the servos, the pen lift, and the emergency stop | Chapter 3 |
| Install or update the firmware | Chapter 4 |
| Set the board for your machine and do the first moves | Chapter 5 |
| Plot from the desktop app | Chapter 7 |
| Plot from AutoCAD or other software | Chapter 8 |
| Understand how the board processes a job | Chapter 9 |
| Find the cause of a fault | Chapter 10 |
| Look up a command, a setting, or an HP-GL instruction | Appendixes A–D |

# Conventions

## Language

The text of this manual follows ASD-STE100, *Simplified Technical English*.[1] STE makes the text easy to read and easy to translate. These are the rules that you will see most:

- A procedure step contains one instruction. Each step starts with a verb in the imperative, for example “Set” or “Do not.”
- A sentence in a procedure has no more than 20 words. A sentence in a description has no more than 25 words.
- The text uses the active voice and the simple tenses.
- A word has one meaning. For example, “enable” always means “supply power to the motor windings,” and “plot” always means “draw a job on the machine.”
- A warning or a caution comes before the step that it applies to.

Technical names, such as setting names, console commands, HP-GL instructions, and file names, are not changed.

## Style

The apparatus of this manual follows *The Chicago Manual of Style*.[2] Chapter and section titles use headline style. Lists of three or more items use the serial comma. Ranges use an en dash, for example IO-0–IO-5. The manual uses the alternative rule for numbers in Chicago 9.3: words for zero through nine in running text, and numerals for 10 and above. Numerals are used for all quantities with a unit, for setting values, and for code. Notes are footnotes in the Chicago notes-bibliography style, and the bibliography is at the end of the manual.

## Safety Notices

**△ WARNING**

A warning tells you about a condition that can cause injury or death. It also tells you about a condition that can make machinery move when you do not expect it.

**CAUTION**

A caution tells you about a condition that can cause damage to equipment, to the material on the bed, or to the settings of the board.

**NOTE:** A note gives information that helps you do a task. It does not tell you about a hazard.

## Typography

All lengths are in millimeters (mm) and all speeds are in millimeters per second (mm/s), unless the text gives a different unit. HP-GL uses *plotter units*: one plotter unit is 0.025 mm, so there are 40 plotter units in 1 mm.

[1] ASD, *ASD-STE100: Simplified Technical English*, issue 8 (Brussels: ASD, 2021).

[2] *The Chicago Manual of Style*, 18th ed. (Chicago: University of Chicago Press, 2024).

**Table 2.** Typographic conventions

| Example | Meaning |
|---|---|
| `set width_mm 600` | A console command, an HP-GL instruction, or text that you type. |
| `draw_mm_s` | A board setting. Appendix B lists all settings. |
| `Firmware/build.ps1` | A file or a directory, relative to the root of the repository. |
| Plotter settings | A menu, a button, a field, or a check box in the desktop app. |
| **M0**, **DI-6** | A connector on the ClearCore. |
| *hard clip* | A term that the glossary defines, at its first use. |

## Related Documents

The file `Firmware/PROTOCOL.md` in the repository is the technical reference for the firmware. It lists the links, the HP-GL instructions, the console commands, and the settings. This manual contains that information. If this manual and `PROTOCOL.md` are different, `PROTOCOL.md` agrees with the firmware. Use it, and report the difference.

Teknic supplies the manuals for the ClearCore and for the ClearPath servos. Use them for the electrical data of the hardware: power supplies, connector pinouts, and the setup of each servo. The bibliography lists them.

CHAPTER 1

# Introduction

## 1.1 What ClearHPGL Is

HP-GL is the language of the HP pen plotters. Many programs can still send a drawing as HP-GL: AutoCAD, CorelDRAW, sign software, and many CAM tools.

ClearHPGL is firmware for the Teknic ClearCore controller. With it, a ClearCore and two ClearPath servos operate as an HP-GL plotter. You point a plotter driver at the USB port, the serial port, or the network address of the board, and the machine draws. You do not need a plug-in, a converter, or special software on the computer.

ClearHPGL has three parts:

- The **firmware**, which runs on the ClearCore. It receives HP-GL, plans the motion, and controls the servos and the pen.
- The **ClearHPGL desktop app** for Windows. It is optional. It opens DXF and HP-GL files, shows a preview, sends jobs, and shows the pen as it moves.
- The **command-line client**, `Firmware/host/clearhpgl_cli.py`. It finds boards on the network, sends files, and gives commands to the console.

## 1.2 Features

- **Standard plotter drivers.** The board operates like an HP 7475A or an HP DraftMaster. HP-GL and HP-GL/2 plotter drivers operate with it without changes.
- **Three links.** A USB virtual COM port, an RS-232 or TTL serial port, and a TCP print port on the network (port 9100).
- **Better path order.** The board can keep a full job before it plots. It joins strokes that touch, draws all shapes inside a contour before the contour, and goes to the nearest shape next.
- **Smooth, coordinated motion.** The two axes move as one machine, with lookahead at each junction. Arcs and circles are true arcs, not short straight lines.
- **Cartesian or CoreXY.** The same firmware drives a machine with one motor per axis, or a CoreXY or H-bot machine with two belts.
- **Safety monitoring.** The board monitors the emergency stop input, the alerts of the servos, and their ready signals. A fault stops the machine and latches.
- **Desktop app.** A preview of DXF and HP-GL files that uses the interpreter of the firmware, a live 3D view of the bed, job monitoring, and machine setup.

## 1.3 How a Job Goes Through the Board

Figure 1.1 shows the path of a job. Each stage is a part of the firmware.

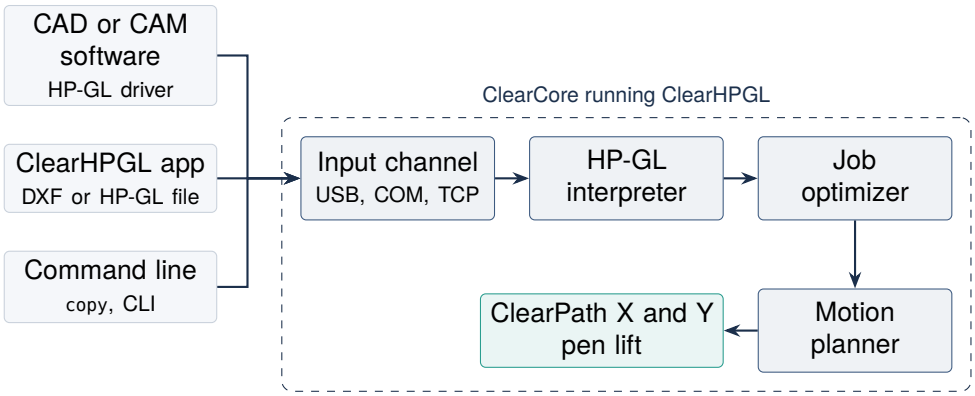



**Figure 1.1.** The path of a job through ClearHPGL

**Input channel**
Receives jobs from all links. One source plots at a time, and the other sources wait. The channel does the flow control of a plotter (XON/XOFF and ENQ/ACK) and answers the `ESC .` device-control sequences. It removes the PCL and PJL wrappers that Windows drivers add.

**HP-GL interpreter**
Changes HP-GL into geometry, as an HP plotter does: scaling, rotation, clip windows, arcs and circles, and HP-GL/2 encoded polylines. It gives a model name that you can set, so 7475A drivers and DraftMaster drivers both operate.

**Job optimizer**
For jobs from CAD software, keeps the full job and then plots it in a better order. Section 9.2 tells you more.

**Motion planner**
Moves the two servos as one coordinated machine, with lookahead and limits for acceleration and for speed at corners.

**Safety**
Monitors the emergency stop, the servo alerts, and the servo ready signals (HLFB) all the time. A fault disables the motors, stops the job, and latches until you clear it.

**Limits and homing**
Optional limit and home switches on the free inputs. A limit switch stops the axis in the same 0.2 ms sample in which it trips. Homing gives plotter 0,0 the same place on the bed each time (chapter 6).

## 1.4 What ClearHPGL Does Not Do

Table 1.1 shows the functions that this version does not have. Do not plan a machine that needs them.

**Table 1.1.** Functions not in firmware version 0.2.0

| Function | Result |
|---|---|
| Text (`LB`) | The board reads the text and does not draw it. The `labels_skipped` counter in `status` shows how many labels it skipped. Set your CAD software to send text as lines ("explode" the text). |
| Area fills | `RA`, `RR`, and `WG` draw only the outline. |
| Line types and pen widths | All lines are solid. `LT` and `PW` have no effect. |
| Soft limits | The board does not limit the travel to the switch positions. The plot area (the hard clip) keeps jobs inside the bed. |
| Pen changes | The machine has one pen. `SP` with a number above 1 does not stop for a pen change. |
| Drag knives | The board does not offset the path for a tangential or drag knife. |
| Lasers | Laser control is a plan for the future. Chapter 2 tells you why you must not connect a laser now. |

Chapter 2

# Safety

Read this chapter before you connect power to the machine. The servos of a plotter are strong and fast. They can cause injury when they move.

## 2.1 Status of the Software

**△ WARNING**

ClearHPGL is experimental. It is not certified for production or for applications where safety is critical. Do not use it as the only safety function of a machine. Install a physical emergency stop that removes power from the motors.

The emergency stop input of the board (**DI-6**) stops the job and disables the motors through the firmware. Firmware can fail. A hardware emergency stop in the motor power circuit does not depend on the firmware. You must have both.

## 2.2 Motion That You Do Not Expect

**△ WARNING**

The motors enable automatically when a job arrives (setting `auto_enable` 1). A job from a computer on the network can start the machine at any time. Keep your hands and tools out of the work area when the board has power and a computer is connected.

**△ WARNING**

Without limit switches, the machine has no protection at the ends of its travel. It goes to the coordinates that the job gives, inside the plot area that you set. If the plot area is larger than the mechanics, the carriage can hit the end stops. Set `width_mm` and `height_mm` to the travel that is really available. Limit switches (chapter 6) stop the axis with a ramp: they are not an emergency stop.

- Before you plot, look at the preview in the desktop app. Make sure that the drawing is in the plot area.
- Before you change the steps per revolution, the travel per revolution, or the direction of an axis, do a test move at low speed. Chapter 5 gives the procedure.
- Keep the work area clear of persons who do not operate the machine.

## 2.3 Faults and the Emergency Stop

The board stops the machine when one of these conditions occurs:

- The emergency stop input **DI-6** opens (with the default setting `estop_di6` 1).
- A ClearPath servo reports an alert or a fault.
- The ready signal (HLFB) of a servo is not present when the motors enable (with `hlfb_check` 1).

When a fault occurs, the board disables the motors, drops the remainder of the job, and latches the fault. The servos stop without control of deceleration. The pen can stay on the material. To continue, remove the cause, then clear the fault with the `clear` command or with Clear fault in the desktop app.

**CAUTION**

When the motors disable, the axes are free. On a vertical machine or a machine on a slope, gravity can move an axis. Make sure that the axes cannot fall or move when the motors have no power.

**NOTE:** If you do not connect an emergency stop to **DI-6**, set `estop_di6` to 0. If you do not do this, the board stays in the fault state and does not plot. A hardware emergency stop in the motor power circuit is still necessary.

## 2.4 Lasers

**△ WARNING**

Do not connect a laser to the pen output. The pen output turns on and off with the timing of a pen: it has settle delays and no control of power. The firmware has no interlocks for an enclosure, a lid, or air assist. A laser on this output can cause fire, burns, and damage to the eyes.

Laser control is a plan for a future version. It will need control of the power by the speed of the path, and interlocks that can stop the beam.

## 2.5 Electrical Safety

**△ WARNING**

Disconnect all power before you connect or disconnect the motors, the pen lift, or the emergency stop. The ClearPath servos and their power supplies have dangerous voltages. Obey the instructions of Teknic for the hardware.

**CAUTION**

The board writes its settings to nonvolatile memory. It can do this only when the supply voltage is above the undervoltage limit of the ClearCore. Do not change settings while the ClearCore gets its power only from the USB cable.

CHAPTER 3

# Hardware and Connections

## 3.1 Parts

Table 3.1 lists the parts of a ClearHPGL plotter. The mechanics (the frame, the guides, and the belts or screws) are your design. The firmware drives two axes and a pen lift, so the plot area is the travel that your mechanics give.

**Table 3.1.** Parts of a ClearHPGL plotter

| Part | Function and notes |
|---|---|
| Teknic ClearCore | Runs the firmware. Needs its own 24 V DC supply. |
| Two ClearPath-SD servos | Step-and-direction servos for X and Y. Each one needs its HLFB (ready) output connected. |
| Pen lift | A solenoid or a relay on a digital output, or a third ClearPath servo on **M2**. |
| Emergency stop | A normally closed contact on **DI-6**, and a separate contact that removes power from the motors. |
| Network cable or USB cable | The link to the computer. A serial cable on **COM-0** or **COM-1** is also possible. |
| Windows computer | For the desktop app and the CAD software. The app is optional. |

## 3.2 Connector Map

Table 3.2 shows what each connector of the ClearCore does with the default settings.

## 3.3 ClearPath Servos

Connect the X servo to **M0** and the Y servo to **M1**. Use the ClearPath MSP software from Teknic to set each servo before you connect it.

**Procedure: Prepare a ClearPath-SD servo**

1. Connect the servo to the computer with its USB cable, and start ClearPath MSP.
2. Set the input format to step and direction.
3. Set the input resolution (the steps per revolution). Use the same value later in `steps_rev_x` or `steps_rev_y`. The default is 800.
4. Set the HLFB output to a mode with a PWM output, with a carrier frequency of 482 Hz. The ClearCore examples from Teknic use "ASG-Position w/Measured Torque."
5. Tune the servo for the load of its axis, as the ClearPath manual tells you.

**Table 3.2.** ClearCore connectors used by ClearHPGL

| Connector | Function | Setting |
|---|---|---|
| **M0** | X axis (CoreXY: motor A), ClearPath, step and direction | `steps_rev_x`, `mm_rev_x`, `dir_x`, `kinematics` |
| **M1** | Y axis (CoreXY: motor B), ClearPath, step and direction | `steps_rev_y`, `mm_rev_y`, `dir_y`, `swap_xy` |
| **M2** | Pen lift servo (optional) | `pen_mode` 1 and the `z_*` settings |
| **IO-0** | Pen lift output (default) | `pen_mode` 0, `pen_io`, `pen_active_high` |
| **DI-6** | Emergency stop input | `estop_di6` |
| **DI-7**, **DI-8**, **A-9**–**A-12**, **IO-0**–**IO-5** | Limit and home switches (optional, chapter 6) | `x_limit_plus` and the other switch inputs |
| **COM-0**, **COM-1** | HP-GL serial port (off by default) | `com_port`, `com_baud`, `com_handshake`, `com_rs232` |
| USB | HP-GL and the setup console | none |
| Ethernet | HP-GL (TCP 9100), console (TCP 9300), discovery (UDP 9302) | `net_mode`, `ip`, `netmask`, `gateway` |

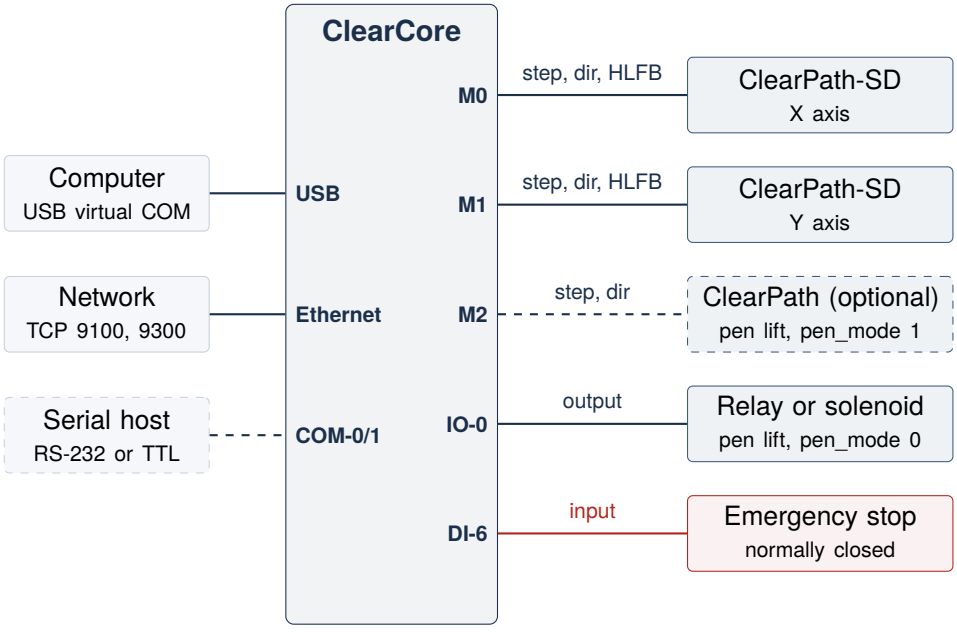



**Figure 3.1.** Connections of a ClearHPGL plotter. Dashed parts are optional.

6. Save the configuration to the servo.

The firmware reads the HLFB signal of each servo. When the motors enable, the board waits for HLFB from all servos. If HLFB does not come, the board disables the motors and reports a fault. Set `hlfb_check` to 0 only if your servos do not supply HLFB.

> **NOTE:** Use the same steps per millimeter on X and Y. The planner uses the scale of the X axis for the speed and the acceleration of the path. Arcs operate as true arcs only when the two scales are the same.

## 3.4 CoreXY and H-Bot Machines

In a *CoreXY* or an H-bot machine, the two motors do not each drive one axis. Two belts move the head together: when both motors turn the same direction, the head moves in X; when they turn in opposite directions, it moves in Y. One motor alone moves the head diagonally.

Set `kinematics` to 1 for these machines. The board then plans and draws in X and Y as usual, and sends these step counts to the motors:

| Connector | Motor | Steps |
|---|---|---|
| **M0** | A | X + Y |
| **M1** | B | X − Y |

- Both belts run on pulleys with the same pitch diameter. Set it in `pulley_dia_mm`. The travel of the belt for one revolution is $\pi$ multiplied by the pitch diameter; `mm_rev_x` and `mm_rev_y` are not used. The default, 12.732, 4 mm, is a 20-tooth GT2 pulley (40 mm per revolution).
- The pitch diameter is the number of teeth multiplied by the belt pitch, divided by $\pi$. For example, a 16-tooth GT2 pulley (2 mm pitch) has a pitch diameter of $32/\pi = 10.186$ mm.
- Both motors must have the same steps per revolution. The board refuses `kinematics` 1 if `steps_rev_x` and `steps_rev_y` are different.
- On a 45° move, one motor does all the work. It turns 1.41 times faster than the head moves, and accelerates 1.41 times faster. Set `motor_max_mm_s` to the highest belt speed that your motors can hold. The board then decreases the speed of each segment that would make a motor go faster.
- `corner_mm_s` limits the change of speed of each motor at a corner.

The direction of each motor and the sequence of the belts change how the head moves. The settings `dir_x`, `dir_y`, and `swap_xy` (exchange X and Y) correct all the possible layouts. You do not have to change the wiring. Section 5.5.1 gives the procedure.

## 3.5 Pen Lift

The firmware has two types of pen lift. Select one with `pen_mode`.

### 3.5.1 Digital Output (pen_mode 0)

The default. A digital output (**IO-0** by default) operates a relay or a solenoid. With `pen_active_high` 1, the output is on when the pen is down. After the board lowers or raises the pen, it waits for the settle time (`pen_down_ms`, default 150 ms; `pen_up_ms`, default 100 ms) before the axes move.

**CAUTION**

Examine the current rating of the ClearCore output in the ClearCore manual. If the solenoid needs more current, use a relay. Install a flyback diode across a coil that has no internal diode.

### 3.5.2 Servo on M2 (pen_mode 1)

A third ClearPath servo on **M2** moves the pen to two positions: `z_up_steps` (default 0) and `z_down_steps` (default 800). Its speed and acceleration are `z_vel_steps` and `z_accel_steps`.

**CAUTION**

The M2 servo has no home position. Before you supply power, put the pen at the pen-up position. The board uses the position at power-up as step 0.

## 3.6 Emergency Stop

Connect a normally closed emergency stop contact to **DI-6**. With the default `estop_di6` 1, the board sees a fault when the input is low (the contact is open, or a wire is broken). Table 3.3 shows the three values.

**Table 3.3.** Values of `estop_di6`

| Value | Result |
|---|---|
| 0 | The board does not monitor **DI-6**. Use only if no emergency stop is connected. |
| 1 | Fault when **DI-6** is low. Use with a normally closed contact (default). |
| 2 | Fault when **DI-6** is high. Use with a normally open contact. |

## 3.7 Links to the Computer

**USB**

The board shows as a virtual COM port. It receives HP-GL at any baud rate. Type `menu` on a line by itself to open the setup console.

**Ethernet**

The board uses DHCP by default. If DHCP fails, the address is 192.168.0.110. It receives HP-GL on TCP port 9100 and gives the console on TCP port 9300.

**COM-0 or COM-1**

An RS-232 or TTL serial port for HP-GL. It is off by default. Set `com_port`, `com_baud`, `com_handshake`, and `com_rs232`, then restart the board.

CHAPTER 4

# Install the Firmware

This chapter tells you how to load the firmware into the ClearCore. The easiest way is from the desktop app (section 4.2). To build the firmware from its source, use the procedure in section 4.3.

## 4.1 What You Need

- A USB cable from the computer to the ClearCore.
- A 24 V supply for the ClearCore. The board must not get its power only from USB when it writes its settings.
- From the app: the ClearHPGL desktop app. It carries the firmware and the flash tool in its `firmware` directory.
- To build from the source: a Windows computer with Microchip Studio 7 (formerly Atmel Studio 7) and the ClearHPGL source repository. The build script uses the ARM compiler and the `make.exe` of Microchip Studio. The repository contains the flash tools: `tools/bossac/bossac.exe` and `tools/flash_clearcore.cmd`, from Teknic.

## 4.2 Update from the App

The desktop app carries the firmware that it was built with. It can put that firmware on the ClearCore over USB. You do not need Microchip Studio or a command line.

**△ WARNING**

Updating restarts the board. Make sure that no job is plotting and that the work area is clear. If the motors are enabled, the update disables them first.

**Procedure: Update the firmware from the app**

1. Connect the ClearCore to the computer with the USB cable.
2. In the app, select Plotter > Update board firmware…. If the app is connected to the board over USB, it disconnects first.
3. In USB port, select the ClearCore. The window shows the firmware on the board and the firmware that the app carries (figure 4.1).
4. Click Update to (or Reinstall) and wait. Do not disconnect the cable.
5. Make sure that the window shows “Firmware ... is running.”
6. Click Close. The app connects to the board again.

The update keeps all settings of the board: the plot area, the speeds, and the network address. They are in a part of the memory that the update does not erase. The update does not start while a job plots.

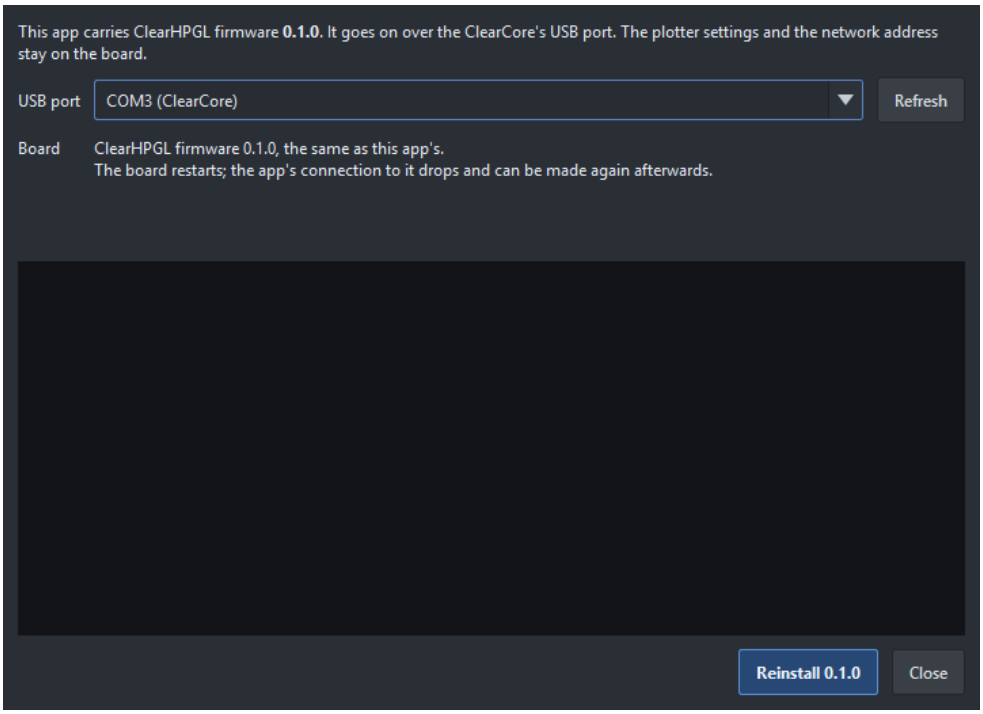



**Figure 4.1.** Update board firmware, with the board read on COM3

> **NOTE:** If the update stops (for example, the cable is disconnected), the board can have no firmware. Push the reset button of the ClearCore two times quickly, so that the IO-0 to IO-5 lights move in sequence. Then click Refresh and update again.

## 4.3 Build and Load the Firmware

**△ WARNING**

Loading firmware restarts the board. Make sure that no job is plotting and that the work area is clear before you start.

**Procedure: Build the firmware and load it into the ClearCore**

1. Connect the ClearCore to the computer with the USB cable.
2. Close all programs that use the USB COM port of the board: a serial terminal, the desktop app, or a CAD plotter driver.
3. Open PowerShell in the root directory of the repository.
4. Type this command and press Enter:

```
./Firmware/build.ps1 -Flash
```

5. Wait. The script builds LwIP, libClearCore, and the firmware. Then it finds the board, puts it into its bootloader, and writes the firmware.

6. Make sure that the output contains the line `Verify successful`.

Without `-Flash`, the script only builds. The firmware files are in `Firmware/ClearCore-Firmware/Release/`: `ClearHPGL_Firmware.bin` and `ClearHPGL_Firmware.uf2`. To load a `.bin` file that you received from a different person, use this command:

```
tools\flash_clearcore.cmd path\to\ClearHPGL_Firmware.bin
```

**NOTE:** The flash script looks for the board on its normal USB port. Then it waits for the bootloader port. The two ports have different COM numbers. This is correct. If the script cannot find the board, push the reset button of the ClearCore two times quickly to start the bootloader, then run the script again.

## 4.4 Make Sure That the Firmware Operates

**Procedure: Examine the firmware after you load it**

1. Open the USB COM port of the board in a serial terminal (for example PuTTY). The baud rate has no effect.
2. Type `menu` on a line by itself and press Enter. The console shows its list of commands.
3. Type `version` and press Enter. Make sure that the version is the one that you loaded.
4. Type `status` and press Enter. Make sure that `fault=none`.
5. Type `exit` and press Enter to close the console.

The settings stay in the board when you load new firmware. A new firmware version that adds a setting gives it its default value.

## 4.5 Settings of Other Firmware

The settings of ClearHPGL have their own marker (`CHPG`) in the nonvolatile memory. If the board ran ClearCoreROS or ClearAI before, and ClearHPGL has no saved settings yet, ClearHPGL keeps the network settings of that firmware, so the board keeps its IP address. All other settings start at their defaults.

CHAPTER 5

# First Setup

Do the procedures in this chapter in the given sequence when you start a new machine. You can do them in the console of the board, or in Plotter settings in the desktop app (section 7.8).

## 5.1 Open the Console

The console is a text interface for setup and for diagnosis. It is available on two links:

**USB**
Open the USB COM port in a serial terminal. Type `menu` on a line by itself and press Enter. Type `exit` to go back to HP-GL mode. While the console is open, the USB port does not receive HP-GL.

**TCP port 9300**
Connect a terminal (for example PuTTY in "raw" mode, or `ncat`) to port 9300 of the board. The console accepts one client at a time. The desktop app uses this port when it is connected over Ethernet.

Each command ends with a line that starts with `ok` or `error:`. Appendix A lists all commands. These are the commands that you use most:

```
status                  # state, position, pen, counters, and links
config                  # all settings and their values
set <key> <value>       # change a setting and save it
enable / disable        # supply or remove motor power
goto <x_mm> <y_mm>      # pen-up move to a position
pen up / pen down       # test the pen lift
origin                  # the present position becomes plotter 0,0
clear                   # clear a latched fault
```

### 5.1.1 When a Setting Takes Effect

Each setting has a rule for when you can change it. The `config` command shows the rule at the end of the line. The desktop app shows it as a label next to the field.

## 5.2 Set the Kinematics

Set the kinematics first. It is the first setting in `config` and in the app, because it selects the mechanics settings that apply.

- `kinematics` 0, Cartesian (default): one motor for each axis. **M0** moves X and **M1** moves Y.
- `kinematics` 1, CoreXY or H-bot: two belts move the head together (section 3.4).

**Table 5.1.** When a setting can change

| Mark in `config` | Label in the app | Rule |
|---|---|---|
| (none) | (none) | Live. The change takes effect immediately, also during a job. |
| `[idle]` | between jobs | The board refuses the change while a job plots. The motors can stay enabled. |
| `[disabled]` | motors disabled | The board refuses the change while the motors are enabled. Use `disable` first. The app does this for you. |
| `[restart]` | after restart | The board saves the change. It takes effect after `restart`. |

**Procedure: Set the kinematics**

1. Type `disable`.
2. Type `set kinematics 0` for a Cartesian machine, or `set kinematics 1` for a CoreXY or H-bot machine. The present position becomes the origin.

## 5.3 Set the Mechanics

The board must know how far the head moves for one revolution of a motor.

- `steps_rev_x`, `steps_rev_y`: the steps per revolution of **M0** and **M1**. Use the input resolution that you set in ClearPath MSP. Default 800.
- Cartesian: `mm_rev_x`, `mm_rev_y`: the travel of the axis for one revolution of the motor, in mm. Default 5.0. For a lead screw, it is the lead of the screw. For a belt, it is the number of pulley teeth multiplied by the belt pitch.
- CoreXY: `pulley_dia_mm`: the pitch diameter of the pulleys, in mm, for both motors. Default 12.732, 4 mm (a 20-tooth GT2 pulley). The pitch diameter is the number of teeth multiplied by the belt pitch, divided by $\pi$.

**Procedure: Set the mechanics of a CoreXY machine**

1. Type `disable`.
2. Type `set steps_rev_x 800` and `set steps_rev_y 800`. Use the value from ClearPath MSP. The two values must be the same.
3. Type `set pulley_dia_mm 12.7324`. Use the pitch diameter of your pulleys.
4. Type `config`, and make sure that the values are correct.

**Procedure: Set the steps and the travel per revolution (Cartesian)**

1. Type `disable`.
2. Type `set steps_rev_x 800`. Use the value from ClearPath MSP.

3. Type `set mm_rev_x 40`. Use the travel of your X axis.
4. Do steps 2 and 3 again for Y, with `steps_rev_y` and `mm_rev_y`.
5. Type `config`, and make sure that the four values are correct.

**NOTE:** On a Cartesian machine, use the same steps per millimeter on X and Y. If the two axes are different, the speeds use the X scale, and arcs are drawn as short straight lines. On a CoreXY machine, the scales are always the same.

## 5.4 Set the Plot Area and the Origin

The *plot area* is `width_mm` by `height_mm` (default 400 by 300). It is the *hard clip*: the board does not draw outside it. Plotter 0,0 is the lower-left corner of the plot area. The position of the pen at power-up is 0,0, until you set a different origin.

**△ WARNING**

Without limit switches, the board does not know where the end stops of your machine are. Set the plot area to the travel that is really available from the origin. If the area is too large, the carriage can hit the end of its travel. Chapter 6 tells how to add limit and home switches.

**Procedure: Set the plot area and the origin**

1. Measure the travel of each axis from the corner that you want as the origin.
2. Type `set width_mm 600`. Use your X travel.
3. Type `set height_mm 300`. Use your Y travel.
4. Move the pen to the lower-left corner of the plot area. Use `goto` or the Jog tab of the app.
5. Type `origin`. The present position becomes plotter 0,0.

**NOTE:** The board does not keep the origin when it restarts. After power-up, the origin is where the pen is. Move the pen to the corner before you supply power, or set the origin again.

## 5.5 Make Sure That the Axes Move Correctly

**△ WARNING**

Keep your hands out of the work area. The axes move when you send the next commands.

**Procedure: Do the first moves**

1. Type `set draw_mm_s 20` and `set travel_mm_s 20` to get a low speed.
2. Type `enable`. Make sure that the reply is `ok`.
3. Type `goto 50 0`. The pen must move 50 mm in the positive X direction (to the right on a standard plotter).

4. Measure the distance. If it is not 50 mm, examine `steps_rev_x` and `mm_rev_x`.
5. If the axis moved in the wrong direction, type `goto 0 0`, `disable`, then `set dir_x -1`.
6. Do steps 3 to 5 again for Y with `goto 0 50` and `dir_y`.
7. Type `goto 0 0` to go back to the origin.

### 5.5.1 CoreXY and H-Bot Machines

On a CoreXY or an H-bot machine (section 3.4), set the kinematics first. Then find the correct directions with this procedure.

**△ WARNING**

With the wrong kinematics or directions, the head can move in a direction that you do not expect. Use a low speed and short moves, and keep the head away from the ends of its travel.

**Procedure: Set the kinematics and the directions of a CoreXY machine**

1. Type `disable`.
2. Make sure that `kinematics` is 1 and that the steps per revolution and `pulley_dia_mm` are set.
3. Type `set draw_mm_s 20` and `set travel_mm_s 20`.
4. Type `enable`, then `goto 20 0`.
5. Look at the movement of the head, and find it in table 5.2.
6. Type `disable`, then do the correction in the table.
7. Type `origin`, and do steps 4 to 6 again until the head moves 20 mm to the right.
8. Type `goto 20 20`. The head must move 20 mm up (positive Y). If it moves down, type `disable`, set `dir_y` to the other value, type `origin`, and do this step again from `goto 20 0`.

**Table 5.2.** Corrections for a CoreXY machine after `goto 20 0`

| The head moves | Correction |
|---|---|
| Right (positive X) | Correct. Go to the last step. |
| Left (negative X) | Set `dir_x` to the other value (1 or -1). |
| Up or down (Y) | Set `swap_xy` to the other value (0 or 1). |
| Diagonally | `kinematics` is not 1, or the belts are not a CoreXY or H-bot layout. |

## 5.6 Set the Pen Lift

**Procedure: Set and test the pen lift**

1. Type `disable`.
2. Type `set pen_mode 0` for a digital output, or `set pen_mode 1` for a servo on **M2**.
3. For a digital output, set `pen_io` (0 to 5) and `pen_active_high`.
4. For a servo on **M2**, set `z_up_steps` and `z_down_steps`.
5. Type `enable`.
6. Type `pen down`. Make sure that the pen goes down.
7. Type `pen up`. Make sure that the pen goes up.
8. If the pen moves before it is fully down or up, increase `pen_down_ms` or `pen_up_ms`.

## 5.7 Set the Speeds

Table 5.3 shows the motion settings. All of them are live, so you can change them during a test plot and see the result.

**Table 5.3.** Motion settings

| Setting | Default | Function |
|---|---|---|
| `draw_mm_s` | 100 mm/s | Speed with the pen down. |
| `travel_mm_s` | 200 mm/s | Speed with the pen up. |
| `max_mm_s` | 250 mm/s | Highest pen-down speed that a job can ask for with `VS`. |
| `job_speed` | 1 | 1: the speed in the job (`VS`) sets the pen-down speed. 0: always use `draw_mm_s`. |
| `accel_mm_s2` | 2000 mm/s$^2$ | Acceleration along the path. It also limits the speed on curves. |
| `corner_mm_s` | 20 mm/s | Largest change of speed of one motor at a corner. A larger value gives faster corners and more shock to the machine. |
| `motor_max_mm_s` | 0 (none) | Highest speed of a motor, as belt travel. For CoreXY, where a motor can turn 1.41 times faster than the head moves. |

Start with low values. Increase them until the machine is fast and the lines are still clean. A heavy carriage or a long belt needs a lower acceleration.

## 5.8 Set the Network

The board uses DHCP by default. If it gets no address, it uses 192.168.0.110. For a fixed address, do this procedure.

**Procedure: Set a static IP address**

1. Type `set net_mode 1`.
2. Type `set ip 10.168.41.5`. Use your address.
3. Type `set netmask 255.255.255.0`.
4. Type `set gateway 10.168.41.1`. If no other network must reach the board, you can keep 0.0.0.0.
5. Type `restart`. The board restarts with the new address.
6. From the computer, type `ping 10.168.41.5` to make sure that the board replies.

To go back to DHCP, type `set net_mode 0` and `restart`.

## 5.9 Do a Test Plot

Send a small square to the board. On Windows, save this text in a file with the name `square.plt`:

```
IN;SP1;PU400,400;PD2400,400,2400,2400,400,2400,400,400;PU0,0;
```

This draws a 50 mm square that starts 10 mm from the origin. Send it to the USB port (for example COM3) with this command in `cmd.exe`:

```
copy /b square.plt COM3
```

The motors enable automatically, the square plots, and the pen goes back to 0,0. Measure the square. Each side must be 50 mm.

> **NOTE:** The board keeps a job from the USB port until the port is quiet for 2 s, and then plots it in a better order (section 9.2). Thus the plot starts approximately 2 s after the file is sent.

CHAPTER 6

# Limits and Homing

Limit switches and homing are optional. Without them, the board does not know where the ends of the axes are: the plot area and the origin are all that keep the pen on the bed (section 5.4). With them:

- A *limit switch* stops the motion of an axis toward the end of its travel. A job that reaches a limit stops, and the board drops it.
- A *home switch* gives the axis a known position. After homing, plotter 0,0 is at the same place on the bed each time.

The switches belong to the axes as HP-GL sees them (X and Y), not to the motors. Thus they operate the same way on a Cartesian machine, on a CoreXY machine, and with `dir_x`, `dir_y`, or `swap_xy`.

**△ WARNING**

A limit switch is not an emergency stop. The board stops the axis with a ramp, so the axis moves past the switch before it stops. Connect an emergency stop to **DI-6** (section 2.3) and keep space for the stop distance after each limit switch.

## 6.1 How the Board Uses the Switches

**Inputs.** Each switch uses one input of the ClearCore: **DI-7**, **DI-8**, **A-9** to **A-12**, or **IO-0** to **IO-5**. The board does not let two switches use the same input, with one exception: the home switch of an axis can use the input of a limit switch of the same axis. Then the axis homes on that limit switch. **DI-6** is for the emergency stop, and the pen output (`pen_io`) cannot be a switch.

**Contacts.** A *normally closed* switch is on at rest and off when it trips. This is the default and the recommended contact: a broken wire reads as a tripped switch, so the axis stops. A *normally open* switch is off at rest. Set its bit in `switch_open` (table B.5).

**Filter.** The board reads the inputs at 5 kHz, in the same interrupt that makes the steps. A switch changes state when its input has the new level for three samples in a row (0.6 ms). The board keeps the position of the first of the three samples. This is the position where the switch tripped.

**Limits.** A tripped limit switch stops only the motion *toward* it. Motion away from it is always possible, so you can move the axis off the switch. The stop is a ramp along the path at three times `accel_mm_s2`. It starts in the same sample in which the switch trips. The distance after the switch is approximately:

$$d = \frac{v^2}{2 \times 3 \times a} + v \times 0.6\ \text{ms} \tag{6.1}$$

where $v$ is the speed and $a$ is `accel_mm_s2`. For example, at 200 mm/s and 2000 mm/s$^2$, $d \approx$ 3.5 mm. At a jog speed of 10 mm/s, $d$ is less than 0.02 mm.

**Table 6.1.** What a limit switch does

| Motion | Result |
|---|---|
| A job, `goto`, or `pen` | The axis stops. The board drops the job and raises the pen. The fault `limit X+ (DI-7)` (for example) latches. The motors stay enabled. The next job waits until you type `clear`. |
| A jog | The jog stops. This is not a fault. A jog toward the tripped limit is refused; a jog away from it is accepted, and clears the fault. |
| Homing | Homing stops with the error `limit X- reached before the home switch` (for example), unless the home switch uses the input of that limit. |

## 6.2 Connect the Switches

**CAUTION**

Disconnect the power of the ClearCore before you connect the switches.

**Procedure: Connect a limit or home switch**

1. Use a switch with a contact for the input voltage of the ClearCore (24 V DC). Use normally closed contacts if possible.
2. Connect the switch between the 24 V supply and the input, as the ClearCore manual shows for a digital input.
3. Put the limit switch where the carriage trips it before the end of the travel. Keep the stop distance $d$ (section 6.1) between the switch and the end.
4. Write down the input of each switch. The wizard finds them again, so a mistake here is not a problem.

## 6.3 Set Up the Switches With the Wizard

The desktop app has a wizard that finds each switch, reads its contact, and proves each one by motion. It sets one axis at a time. Do it again for the other axis.

**Procedure: Set up the switches of an axis**

1. Connect the app to the plotter (section 7.3).
2. Select Plotter > Set up limits and homing….
3. Axis: select X or Y and the switches that the axis has. If the axis homes on a limit switch, select it in Home switch is. Keep the setup jog speed low (10 mm/s). Click Next.

4. Find each switch: click Find for a switch, then trip the switch by hand for approximately one second and release it. The wizard shows the input that changed. Do this for each switch.
5. Contact: make sure that the contact of each switch is correct. The wizard reads it from the level at rest.
6. Direction: click Enable motors. Hold Hold: jog + until a switch stops the axis, then hold Hold: jog -. The plotter stops at the first switch of the axis. If the axis goes toward + and stops on the - limit, the wizard asks a question:
   - If the head went to the - end, click The head went the other way: reverse the axis. The wizard changes `dir_x` or `dir_y` and disables the motors. Enable them and jog again.
   - If the head went to the + end, click I tripped them the wrong way round: swap them.
7. Limits: the wizard writes the limits to the plotter. Hold the jog until each limit stops the axis, then jog back off it. The wizard shows how far the axis moved past each switch.
8. Home: make sure that the home direction is correct. Set the position of the axis at the switch: 0 for a switch at the 0 end of the plot area, or the width (or height) for a switch at the far end. Click Write and home. The plotter homes the axis the number of times that you set, and the wizard shows the spread of the positions.
9. Review: select Home before plotting if the plotter must home before each job after power-up. Click Apply.

> **NOTE:** The Limits and Home steps write their settings to the plotter immediately. Close does not remove them. To remove a switch, set its input to `none` in Plotter settings.

## 6.4 Set Up the Switches in the Console

You can also set the switches with `set`. This example has a normally closed X- limit on **DI-7**, which is also the X home switch, and a normally closed Y- limit and home switch on **DI-8**:

```
set x_limit_minus DI-7
set x_home DI-7
set y_limit_minus DI-8
set y_home DI-8
set x_home_dir -1
set y_home_dir -1
set home_mode 1
home
```

Use `swinfo` to see the levels of the inputs, the state of each switch, the limit stops, and the last homing (appendix A).

## 6.5 Homing

`home` (or Home in the app) homes each axis that has a home switch. `home x` or `home y` homes one axis. For each axis, in the sequence that `home_order` sets, the board:

1. Raises the pen, if it is down.
2. Moves off the home switch, if the switch is tripped.
3. Moves toward the switch at `x_home_fast` until the switch trips. If it does not trip within `x_home_travel` (or 1.2 times the plot area plus 20 mm), homing stops with an error.
4. Moves back until the switch releases, then `x_home_back` more.
5. Moves toward the switch at `x_home_slow` until it trips again.
6. Sets the position where the switch tripped to `x_home_at`. The position is a whole number of steps, so each homing gives the same result.

The head stops a small distance past the switch. Homing lasts for each axis approximately the search distance divided by the search speed, plus a few seconds.

**Procedure: Home before each job**

1. Set up homing (section 6.3).
2. Type `set home_required 1`, or select Home before plotting in the wizard.
3. Send a job. If the axes are not homed, the board homes them first and then plots. While it homes, the state is `homing`.

The board forgets the homed position when you disable the motors, when the emergency stop opens, when a servo has a fault, and when you set the origin. The servos can move when they are disabled, so the position is not sure after that.

**ClearPath hard-stop homing.** With `home_mode 2`, the ClearPath servos home themselves against a hard stop, with no switch. This is for Cartesian machines only, because each servo homes one axis.

**Procedure: Use ClearPath hard-stop homing**

1. In ClearPath MSP, set homing for each servo of X and Y. Set it to home on every enable.
2. Type `set hlfb_check 1`. The board sees the end of homing on HLFB.
3. Type `set x_home_at 0` and `set y_home_at 0`, or the positions where the hard stops are.
4. Type `set home_mode 2`.
5. Type `home`. The board disables the servos, enables them again, and waits up to 120 seconds for HLFB. Then it sets the position.

## 6.6 Homing Errors

**Table 6.2.** Homing errors

| Message | Action |
|---|---|
| `X home switch not found within the search travel` | Make sure that the switch operates (`swinfo`) and that `x_home_dir` goes toward it. Increase `x_home_travel`. |
| `limit X- reached before the home switch` | The home switch is beyond the limit, or `x_home_dir` is wrong. |
| `X home switch did not release` | The switch stays tripped. Examine the switch and its wire. Increase `x_home_back`. |
| `X home switch lost on the slow approach` | The switch did not trip again. Increase `x_home_back`, or examine the switch for bounce. |
| `no X home switch (set x_home)` | `home x` needs a home switch. |
| `ClearPath homing did not finish (HLFB not asserted)` | Examine the homing settings in MSP and the HLFB cable. |

## 6.7 Test Without Switches

`sim` simulates a switch on an input, for a test on the bench. A simulated switch is fixed to the machine: it stays at the same place when homing or `origin` changes the position.

**Procedure: Test homing with a simulated switch**

1. Type `set x_home A-9`, `set x_home_dir -1`, and `set home_mode 1`.
2. Type `sim A-9 x- 30`. A switch is now 30 mm from the head, on the - side of X.
3. Type `enable`, then `home x`.
4. Type `swinfo`. Make sure that the `home` line shows `homed=1` and `last=ok`.
5. Type `sim clear`. Set the settings back.

Chapter 7

# The ClearHPGL Desktop App

The ClearHPGL app is a Windows program for the plotter. It is optional: the board plots without it. With the app, you can:

- Open DXF and HP-GL files, and see a preview of the plot.
- Find problems in a DXF drawing and repair some of them.
- Send a job and follow the pen in a 3D view of the bed.
- Enable, pause, and stop the machine, and move the pen.
- Change the settings of the board.

## 7.1 Install the App

The app is built from the source repository. You need Visual Studio 2022, CMake, and Qt 6 from vcpkg.

**Procedure: Build and install the app**

1. Open PowerShell in the root directory of the repository.
2. Type `./build.ps1 -Install` and press Enter.
3. Wait until the script shows `Done.`
4. Start the app from the Start menu entry ClearHPGL.

`./build.ps1` without `-Install` builds the app in `ProjectData/build/Release/`. The script writes that directory again at each build. The `-Install` copy goes into `Programs/ClearHPGL` in the `%LOCALAPPDATA%` directory. It does not change until you install again. Pin that copy to the task bar.

## 7.2 The Main Window

Figure 7.1 shows the main window. It has four areas:

**Connection**
The link to the board: Transport, the plotter address or port, Discover, and Connect.

**Plotter**
The machine buttons: Enable, Disable, Abort, Pause, Set origin, Clear fault, and Home. Home is available when homing is set (chapter 6). Below them are the Plot tab and the Jog tab.

**Plot area**
A 3D view of the bed, the drawing, and the pen.

**Status bar**
The state of the link and of the machine. Show log opens the log of commands and replies.

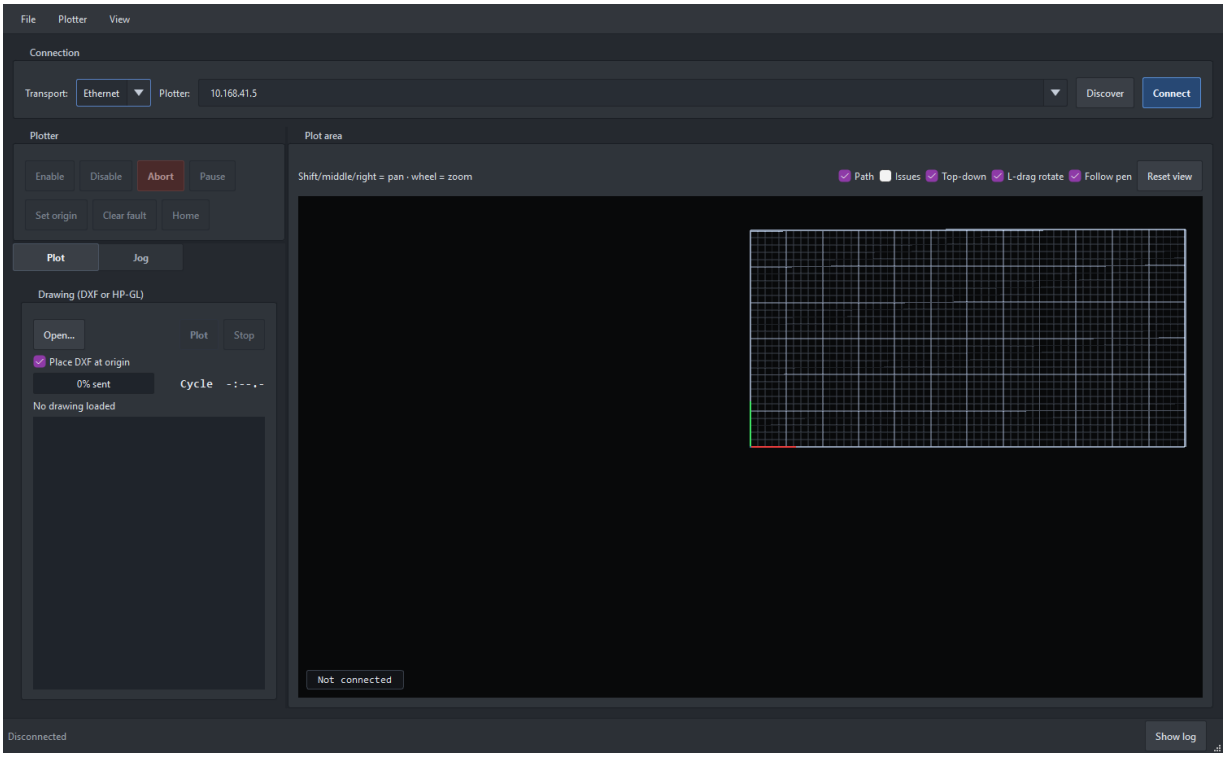



**Figure 7.1.** The main window of the ClearHPGL app, before a connection

## 7.3 Connect to the Plotter

The app connects over Ethernet or over a serial port.

**Procedure: Connect over Ethernet**

1. Set Transport to Ethernet.
2. Click Discover. The app finds the boards on the local network and adds them to the Plotter list.
3. If the board is not found, type its IP address in Plotter.
4. Click Connect.

Over Ethernet, the app uses the console port (TCP 9300) for commands and status, and the HP-GL port (TCP 9100) for jobs. The console port accepts one client. While the app is connected, a terminal cannot use port 9300.

**Procedure: Connect over a serial port**

1. Set Transport to Serial.
2. Click Refresh, then select the COM port of the board.
3. For the USB port, keep any baud rate and No flow control.
4. For **COM-0** or **COM-1**, set the baud rate to `com_baud` and the flow control to `com_handshake`.

5. Click Connect.

When the app is connected, the status bar shows the state of the machine (for example `idle`, `plotting`, `homing`, or `fault`). With homing set, it also shows `homed` or `not homed`, and `ON LIMIT X+` (for example) while a limit switch is tripped. The 3D view shows the pen at its present position.

## 7.4 Open a Drawing

Click Open… on the Plot tab, or use File > Open drawing (DXF / HP-GL)…. The app opens two types of file.

### 7.4.1 HP-GL Files

Files with the extension `.plt`, `.hpgl`, `.hgl`, `.hpg`, or `.prn`. The app sends the file to the board without changes. For the preview, the app uses the HP-GL interpreter of the firmware. Thus the position, the rotation, the scaling, and the clipping in the preview are the same as on the machine.

### 7.4.2 DXF Files

The app reads ASCII DXF files. It reads these entities: `LINE`, `LWPOLYLINE`, `POLYLINE` (also with bulges), `ARC`, and `CIRCLE`. It does not read binary DXF, text, splines, ellipses, or blocks. In your CAD software, explode these objects into lines, arcs, and polylines before you save the DXF.

**CAUTION**

The app uses the `$INSUNITS` variable of the DXF to get the units. If the DXF has no `$INSUNITS`, or the value is 0 (unitless), the app uses inches. Save the DXF with its units set, or the plot can be 25.4 times too large or too small.

The app prepares a DXF for plotting:

- It joins lines whose ends touch into longer paths.
- It plots all shapes inside a closed contour before the contour. Thus holes are cut before the outline frees the part.
- It goes to the nearest shape next, to make the pen-up travel short.
- It sends arcs and circles as true arcs (HP-GL `AA`), not as short straight lines. The machine draws them as smooth curves.

When Place DXF at origin is selected, the app moves the drawing so that its lower-left corner is at plotter 0,0. Clear it to keep the coordinates of the DXF.

## 7.5 The 3D View

The 3D view shows the bed as a grid with lines every 10 mm. The drawing is on the bed; pen-down lines and pen-up moves have different colors.

### 7.5.1 Problems in a Drawing

When Issues is selected, the view shows markers where the drawing can give a bad result. Put the mouse on a marker to read the explanation. The app finds these problems:

**Table 7.1.** Controls of the 3D view

| Control | Function |
|---|---|
| Mouse wheel | Zoom in and out. |
| Shift + drag, middle drag, or right drag | Move the view. |
| Left drag | Turn the view, when L-drag rotate is selected. |
| Path | Show or hide the drawing. |
| Issues | Show or hide the markers for problems in the drawing. |
| Top-down | Look straight down on the bed. |
| Follow pen | Keep the pen in the center of the view while it plots. |
| Reset view | Show the full drawing again. |

- Contours that do not close (open profiles).
- Contours that almost touch across a pen-up move.
- Long pen-down jumps inside a path, and short gaps between long cuts.
- Corners so tight that the machine must slow down a lot.
- Lines that overlap or cross.

Right-click a marker for its menu. Copy explanation and Copy X, Y copy the information. For a DXF, two quick fixes are available:

**Quick fix: close contour**
Closes an open contour with a smooth bridge across the gap.

**Quick fix: connect across pen-up**
Joins two contours that almost touch into one path.

After a quick fix, the app orders the drawing again. Use File > Save DXF geometry… to keep the repaired geometry in a new DXF.

## 7.6 Plot

**△ WARNING**

The machine moves when you click Plot. Keep your hands out of the work area.

**Procedure: Plot a drawing from the app**

1. Connect to the plotter.
2. Open the drawing.
3. Look at the preview. Make sure that the drawing is in the plot area.
4. Put the material and the pen on the machine.
5. Click Plot.

Before the app sends a DXF job, it compares the drawing with the plot area of the board. If shapes are outside the area, the app shows how many, and asks you to continue or to stop. If the emergency stop input is open, the app does not send the job.

During the plot:

- The progress bar shows how much of the job the app has sent.
- The 3D view shows the pen, and paints the lines that are drawn in green.
- The HP-GL text below the progress bar highlights the instruction that the machine draws now.
- Cycle shows the time from Plot to the end of the plot.

The app also follows jobs from other programs. For example, when AutoCAD plots on the USB port, the 3D view shows the pen and the lines that it draws.

### 7.6.1 Pause and Stop

**Pause**

The machine stops at the end of the segments in its queue and raises the pen. The button changes to Resume. Click Resume to lower the pen and continue.

**Stop**

The app stops sending and stops the plot on the board.

**Abort**

The board stops now, drops the job, and raises the pen. Use it for a job from any source.

> △ **WARNING**
>
> Abort, Stop, and Pause are not emergency stops. They operate through the firmware and the link. Use the emergency stop if a person or the machine is in danger.

## 7.7 Move the Pen

The Jog tab moves the pen with the pen up. The machine must not be plotting.

- Go to (pen up): type an X and a Y position in mm, then click Go.
- Jog: set Step (mm), then click X-, X+, Y-, or Y+. The pen does not move out of the plot area.
- Pen up and Pen down: test the pen lift.

To make the present position the origin, click Set origin.

## 7.8 Plotter Settings

Use Plotter > Plotter settings… to change the settings of the board. The app reads the settings from the board and shows them in groups (figure 7.2). Fixed choices are lists.

A label next to a field tells you when the setting can change (table 5.1):

**motors disabled**

The app disables the motors before it applies the change. The motors enable again with the next job.

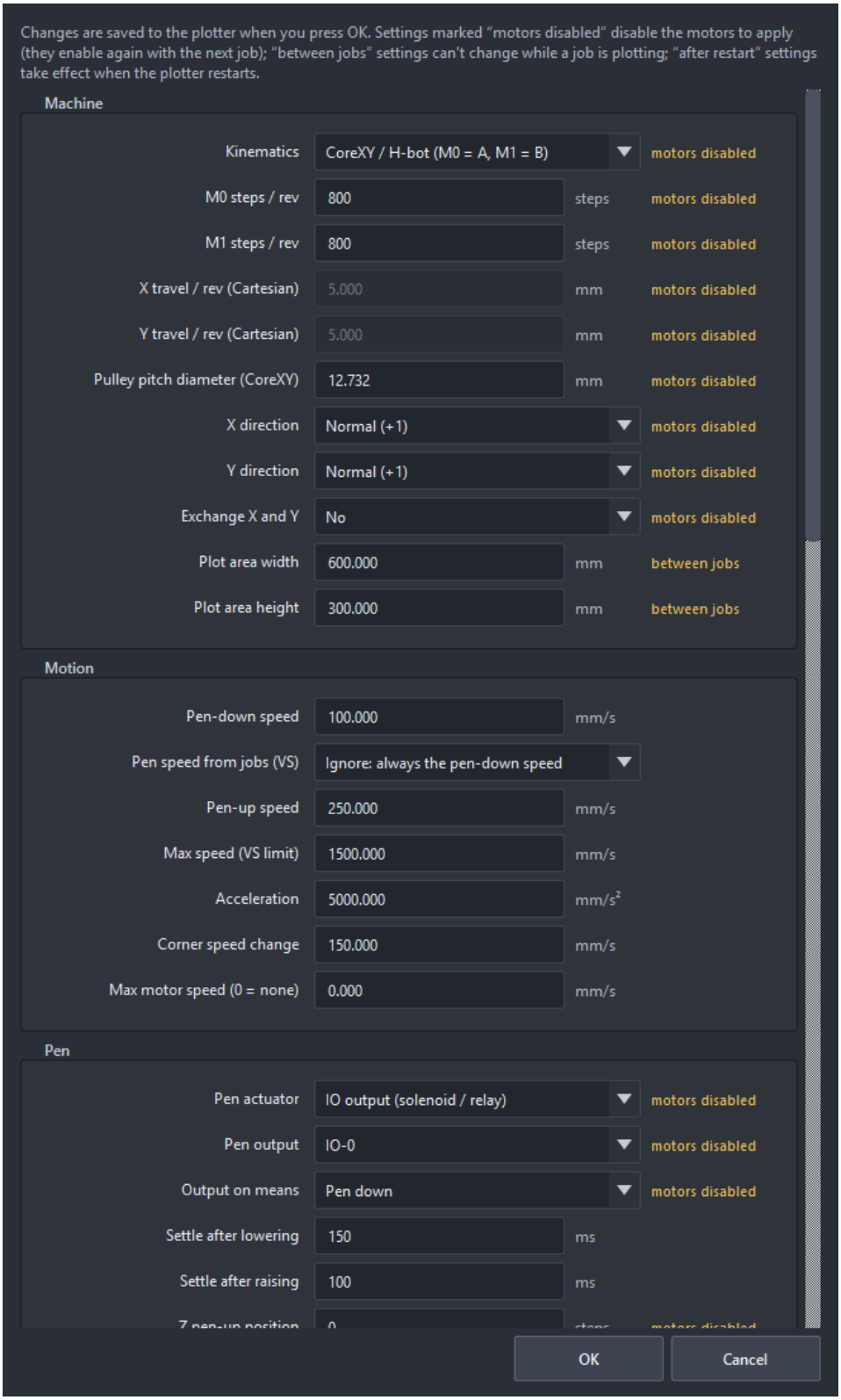



**Figure 7.2.** The Plotter settings dialog, with the factory settings

**between jobs**
The change is not possible while a job plots, but the motors can stay enabled. The plot area has this label.

**after restart**
The board saves the change. It takes effect after the board restarts.

Click OK to send the changes. The app does not send changes while a job plots. If the board refuses a change, the app shows the settings that did not change and the reason. The plot area of the app changes to the plot area of the board.

## 7.9 ClearPath Scope

Plotter > ClearPath scope… (Ctrl+Shift+O) records the signals of the ClearPath servos through the USB port of each servo, the port that ClearPath MSP uses. It shows them in lanes on one time axis (figure 7.3). Use it to see how the X and Y servos follow a plot: the tracking error on each acceleration, the speed, the bus voltage, the temperature, and the HLFB, enable, and alert signals.

The scope does not use the ClearCore. Thus it operates during a plot, and over any link.

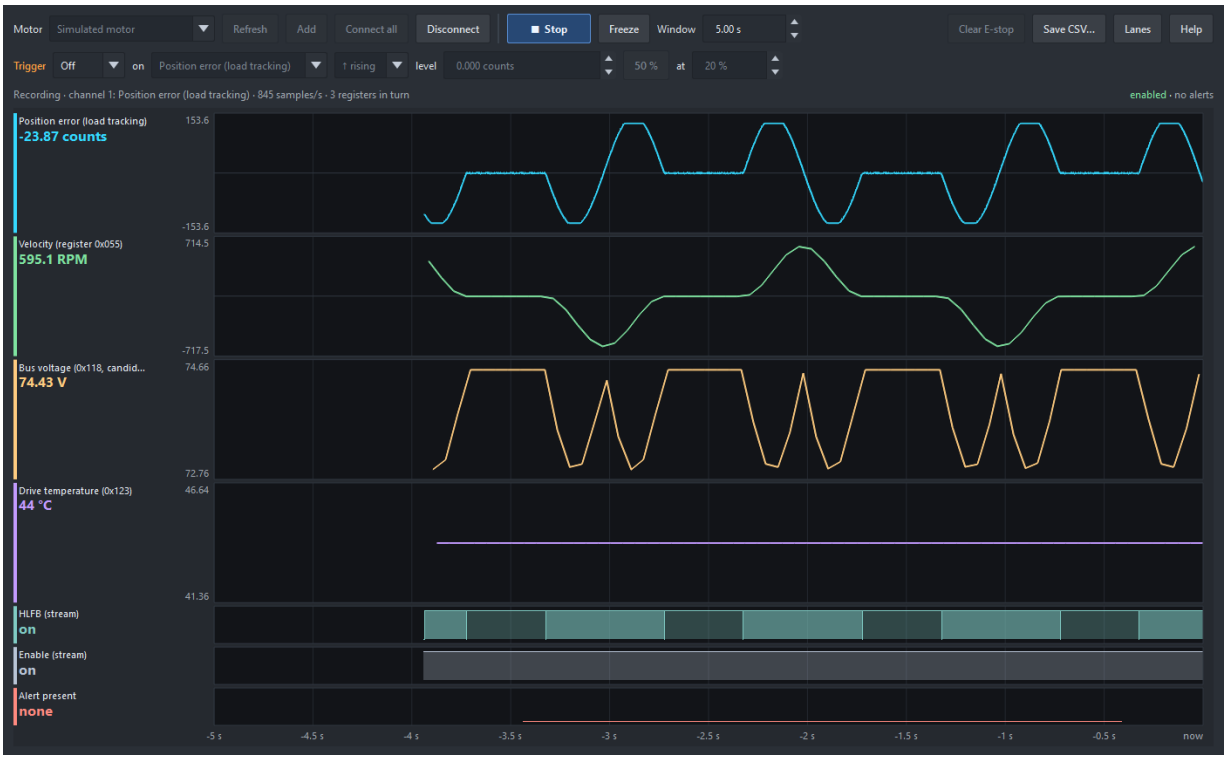



**Figure 7.3.** The ClearPath scope, on its simulated motor

**Procedure: Record a ClearPath servo**

1. Connect the USB port of the servo to the computer.
2. Close ClearPath MSP. One program at a time can use the port.
3. Select Plotter > ClearPath scope….

4. In Motor, select the servo (*ClearPath on COMn*). For more than one servo, click Connect all.
5. Click Connect, then Start.
6. Click Stop to stop the recording, then Disconnect.

- Lanes selects the signals, their sequence, color, and scale.
- Trigger holds the screen on a repeated event, for example HLFB going low at the start of each move.
- Freeze holds the view while the recording continues. Save CSV… saves the recording.
- Help (F1) opens the full description of the scope.
- Without a servo, select Simulated motor to try the scope.

**NOTE:** The scope sets the working monitor channel of the servo while it records, and sets it back when it stops. It saves nothing in the servo, and it never enables or moves the motor.

## 7.10 Limits and Homing

Plotter > Set up limits and homing… opens the wizard that finds the limit and home switches of an axis and proves them by motion. Section 6.3 tells how to use it. The settings are also in the Limits and homing part of Plotter settings.

## 7.11 Other Functions

- File > Export HP-GL… saves the job as an HP-GL file. For a DXF, it is the HP-GL that the app sends.
- View > Command log shows all commands and replies.
- View > Clear pen trace removes the green trace of the last plot.

CHAPTER 8

# Plot from CAD Software

The board operates like an HP plotter. Any program with an HP-GL or HP-GL/2 plotter driver can plot to it. You do not need the desktop app.

## 8.1 AutoCAD

AutoCAD has drivers for the HP DraftMaster plotters. Both of the drivers in table 8.1 plot directly to the board.

**Table 8.1.** AutoCAD drivers for ClearHPGL

| Driver | Output | Setting on the board |
|---|---|---|
| DraftMaster RX 7596B (HP-GL/2), recommended | True arcs and compact encoded polylines. The driver places the plot. | None. |
| Draftmaster 2 (7596A) (HP-GL) | Arcs as short lines. Plotter 0,0 is at the center of the sheet. | `set ident 7596A` |

**Procedure: Set up an AutoCAD plotter for ClearHPGL**

1. In AutoCAD, open the Add-a-Plotter wizard.
2. Select the manufacturer HP and the model DraftMaster RX 7596B.
3. Select the COM port of the board (the USB port or **COM-0**/**COM-1**).
4. In Configure Port, set the flow control to XON/XOFF.
5. Add a custom paper size that is the same as the plot area, with no margins.
6. Plot a small drawing to test the setup.

> **NOTE:** AutoCAD sends its own pen speed (`VS`) from its pen table. To use the speed of the board, type `set job_speed 0`. Then the board always uses `draw_mm_s` for the pen-down speed.

The path order of AutoCAD is not important. By default, the board keeps each job from the USB port and the COM ports, and plots it in a better order (section 9.2).

### 8.1.1 The Model Name

The board replies to the HP-GL `OI` instruction with the model name in `ident` (default `7475A`). Some drivers use this reply. The model name also sets the position of plotter 0,0:

- For `7475A`, `7550A`, and the other small models, plotter 0,0 is the lower-left corner of the plot area.

- For the large DraftMaster models (`758x` and `759x`, for example `7596A`), plotter 0,0 is the center of the plot area. Their drivers send coordinates around the center.

Set `ident` to the model that your driver expects.

## 8.2 Other Software

Use a driver for an HP 7475A, an HP 7550A, or an HP DraftMaster. Most sign, vinyl, and CAM programs have one of these drivers. Then:

- Point the driver at the COM port of the board, or at TCP port 9100 of the board.
- On a COM port, use XON/XOFF flow control.
- Set the paper size to the plot area.
- Set the program to send text as lines. The board does not draw HP-GL text (`LB`).

Windows plotter drivers often put PCL or PJL commands around the HP-GL/2. The board removes them.

## 8.3 Send a File without a Driver

You can send an HP-GL file directly to the board.

**Windows, USB or COM port**

```
copy /b drawing.plt COM3
```

```
nc 10.168.41.5 9100 < drawing.plt
```

**Linux or macOS, network; command-line client, any system with Python**

```
python Firmware/host/clearhpgl_cli.py discover
python Firmware/host/clearhpgl_cli.py --host 10.168.41.5 send drawing.plt
python Firmware/host/clearhpgl_cli.py --serial COM3 send drawing.plt
```

On the network, one connection to port 9100 is one job. The job ends when the connection closes. Appendix D gives more information about the command-line client.

CHAPTER 9

# How the Board Processes a Job

This chapter tells you what the board does with a job. Use it to understand the result of a plot, and to set the board for your work.

## 9.1 One Source at a Time

The board receives HP-GL on three links: USB, COM, and TCP port 9100. One source plots at a time. A source keeps the plotter until its buffer is empty and the job has ended:

- On TCP, the job ends when the connection closes.
- On USB and COM, the job ends when the source is quiet for 2 s.

Then the next source that has data starts. The other sources wait: TCP and USB stop at the full buffer, and COM gets XOFF. The board sends the replies to output instructions (for example `OI` or `OS`) to the source that plots.

If the last instruction of a job has no `;`, the board ends it when the job ends. For example, `PU0,0` at the end of a file operates.

## 9.2 The Job Optimizer

CAD drivers often send a drawing in a bad order: an outline in many short parts, or holes after the outline. The *job optimizer* of the board keeps a full job and then plots it in a better order:

1. It joins strokes whose ends touch into longer paths.
2. It draws everything inside a closed contour before the contour. Thus outlines come last, and a cutter does not free a part before its holes are cut.
3. From the shapes that are ready, it draws the nearest next. It starts a closed contour at its nearest vertex, and an open path at its nearer end.

On the AutoCAD parts that were tested, the optimizer decreased the pen-up travel by 45%–65%.

### 9.2.1 Which Sources the Optimizer Changes

The setting `optimize` selects the sources. Add the values of the sources that you want:

**Table 9.1.** Values of `optimize`

| Value | Source |
|---|---|
| 1 | USB |
| 2 | COM-0 or COM-1 |
| 4 | Network (TCP 9100) |

The default is 3 (USB and COM), because CAD drivers use these links. The desktop app orders its own jobs, so the board does not change network jobs.

### 9.2.2 What the Optimizer Changes

- The plot starts only when the job ends. While the board keeps the job, `status` shows `state=buffering`.
- A pen change or a speed change divides a job into groups. The groups keep their sequence.
- A page break (`PG`) ends a page. The board plots each page when it ends.
- A page can have 8000 points and 500 strokes. A larger job plots in the sequence that it arrives, and the remainder streams through.
- Arcs in a job that the optimizer changes are drawn as short lines (chords).

The `diag` command shows the last job: the number of strokes and paths, and the pen-up travel before and after the optimizer.

## 9.3 Arcs and Circles

The HP-GL instructions for arcs and circles are `AA`, `AR`, `CI`, `EW`, and `WG`. With the pen down, the board draws each one as a *true arc*: one smooth movement on the circle. The speed on an arc is limited, so that the acceleration toward the center stays in `accel_mm_s2`. The `status` command counts the arcs (`arcs=`).

The board draws an arc as *chords* (short straight lines) when:

- the optimizer changes the job (`optimize`),
- the job scales X and Y differently, so the circle is an ellipse,
- the arc goes out of the clip window or the plot area,
- X and Y have different steps per millimeter, or
- the radius is less than a few motor steps.

Chords are 5° by default. The job can set a different angle with the chord parameter, or set a maximum deviation with `CT1`. The minimum is 0.5°.

## 9.4 Speed

- With the pen up, the speed is `travel_mm_s`.
- With the pen down, the speed is `draw_mm_s`. With `job_speed` 1, the `VS` instruction of a job sets it (in cm/s), up to `max_mm_s`. `VS` without a value sets it back to `draw_mm_s`.
- On curves, the board decreases the speed. The acceleration toward the center of the curve stays in `accel_mm_s2`. This applies to true arcs and to curves made of short lines.
- At a corner, the speed of each axis changes by no more than `corner_mm_s`.

The planner looks ahead along the segments in its queue and always plans to stop at the last one. If the computer sends data too slowly, the machine slows down. It does not lose its position.

## 9.5 Coordinates and Clipping

- One plotter unit is 0.025 mm (40 units per mm).
- The plot area is `width_mm` by `height_mm`, from plotter 0,0.
- The board moves the pen up to the edge of the plot area, not past it.
- A pen-down line is clipped to the clip window (`IW`) and to the plot area. The pen goes up where a line leaves the window and down where it comes back.
- A point that is less than half a plotter unit (0.012, 5 mm) outside the plot area is on the edge. A drawing that touches the edge of the plot area plots without a pen lift.
- `RO` rotates the coordinates by 90°, 180°, or 270°.

## 9.6 Pen Changes

The board raises and lowers the pen only when the axes are stopped. After a pen change, it waits for the settle time (`pen_down_ms` or `pen_up_ms`). Between two pen changes, the segments flow through the planner without a stop.

## 9.7 Abort a Job

These actions stop a job, drop the data that the board has kept, and raise the pen:

- The console command `abort`, or Abort in the app.
- The device-control sequence `ESC.K` from the source that plots.

After a fault, the board also drops the job. A TCP job is closed. A serial job is discarded until its source is quiet for 2 s, so the remainder of the job does not plot.

CHAPTER 10

# Troubleshooting

## 10.1 Read the State of the Board

The `status` command shows the state of the machine. The desktop app shows the same data in its status bar and its log. This is an example:

```
state=idle enabled=1 x_mm=0.000 y_mm=0.000 pen=up sel_pen=1 queue=0
planner=0 segments=1767 arcs=550 arc_fallbacks=0 curve_capped=1298
dropped=0 pages=0 inputs=0180 sw=00 homed=0 home_mode=0 estop=ok
fault=none
```

**Table 10.1.** Fields of `status`

| Field | Meaning |
|---|---|
| `state` | `idle`, `buffering` (the optimizer keeps a job), `plotting`, `paused`, `disabled`, `waiting_enable`, or `fault`. |
| `enabled` | 1 if the motors are enabled. |
| `x_mm`, `y_mm`, `pen` | The position of the pen and its state. |
| `queue`, `planner` | Actions that wait in the board, and segments in the planner. |
| `segments`, `arcs` | Lines and true arcs sent to the planner since power-up. |
| `arc_fallbacks` | Arcs that the planner refused and that the board drew as a line. |
| `curve_capped` | Segments that were slower because of a curve. |
| `dropped` | Segments that the board could not plot (for example, zero length). |
| `inputs` | The levels of the inputs, in hexadecimal: bit 0–5 **IO-0**–**IO-5**, bit 7 **DI-7**, bit 8 **DI-8**, bit 9–12 **A-9**–**A-12**. |
| `sw` | Tripped switches: 1 X+, 2 X-, 4 X home, 8 Y+, 10 Y-, 20 Y home (hexadecimal). |
| `homed`, `home_mode` | Homed axes (1 X, 2 Y), and `home_mode`. |
| `estop`, `fault` | The emergency stop input, and the latched fault. |

The second line of `status` shows the links: the bytes received on each link, the source that plots, the IP address, and the counters of HP-GL instructions, unknown instructions, and skipped labels.

## 10.2 Faults

A fault stops the machine (section 2.3). The `fault` field tells you the cause.

To clear a fault, remove the cause, then type `clear` or click Clear fault. The `enable` command also clears it.

**Table 10.2.** Faults

| Fault | Action |
|---|---|
| `estop (DI-6)` | Close the emergency stop. Examine the wiring of **DI-6**. If no emergency stop is connected, set `estop_di6` 0. |
| `motor fault on M0 (X)`, `motor fault on M1 (Y)` | Examine the servo with ClearPath MSP: tracking error, overload, or power. Then type `clear`. |
| `limit X+ (DI-7)` (and the other limits) | A job moved into a limit switch. Jog the axis away from the switch (the app or `jog`), find why the job went there (plot area, origin, homing), then type `clear`. |
| Homing errors | Table 6.2. |
| `HLFB not asserted` | A servo does not supply its ready signal. Examine its power, its cable, and its HLFB setting in MSP. Set `hlfb_check` 0 only if your servos have no HLFB. |
| `pen Z move timed out` | The pen servo on **M2** did not finish its move in 5 s. Examine the servo and `z_vel_steps`. |

## 10.3 Diagnostics

The `diag` command shows information about the motion. The command `diag reset` sets the timing counters to zero first.

```
bursts=0 (samples asking > 16 steps) excess_carried M0=0 M1=0
timing: max_irq_off=0 us (QueueLinear/QueueArc) max_update_gap=212 us ...
step interrupt: max=93 us (arc start)  slow(>150us of 200)=0
controller x=68900 y=15239  pulses_sent M0=68900 M1=15239
optimizer: idle  last job: 3 strokes -> 3 paths, pen-up travel 136 -> 127 mm
```

**bursts**
Samples that asked for more than 16 steps. It must be 0.

**step interrupt**
The longest time of the step interrupt, and how many times it used more than 150 µs of its 200 µs period. If the interrupt is too long, a motor can jump. A value near 200 µs is a problem.

**controller, pulses_sent**
The position of the planner, and the pulses sent to each motor since power-up or the last `origin`. The value "as x, y" changes the pulses back to X and Y (for CoreXY, X = (A + B) / 2 and Y = (A − B) / 2). It must be the same as the position of the planner. The line also shows `kinematics` and `swap_xy`.

**optimizer**
The last job: strokes, paths, and the pen-up travel before and after the optimizer.

## 10.4 Problems and Remedies

**Table 10.3.** Problems and remedies

| Problem | Possible cause and remedy |
|---|---|
| Nothing plots from the USB port | The console is open: type `exit`. The optimizer keeps the job until the port is quiet for 2 s: wait. A fault is latched: read `status`. |
| The state stays `waiting_enable` | `auto_enable` is 0. Type `enable`, or set `auto_enable` 1. |
| The motors do not enable | The emergency stop is open, or HLFB is missing. Read the fault. |
| The board is not found on the network | The board uses 192.168.0.110 if DHCP fails. Make sure that the computer is on the same subnet. Type `status` on the USB console to read the `ip` field. |
| A terminal cannot connect to port 9300 | The console accepts one client. Close the desktop app or the other terminal. |
| The plot has the wrong size | Examine `steps_rev_x`, `mm_rev_x`, `steps_rev_y`, and `mm_rev_y`. For a DXF, examine its units (`$INSUNITS`). |
| The plot is mirrored | Set `dir_x` or `dir_y` to -1. |
| The plot is at the wrong position | Set the origin again. For an AutoCAD DraftMaster driver, examine `ident` and the paper size. |
| Parts of the drawing are missing | The drawing is outside the plot area. Text (`LB`) is not drawn: send text as lines. |
| The pen moves before it is down | Increase `pen_down_ms`. |
| Corners are rough, or the machine shakes | Decrease `accel_mm_s2` or `corner_mm_s`. |
| The machine is slow on small curves | This is correct: the speed on a curve is limited by `accel_mm_s2`. Increase it if the machine can accelerate more. |
| Arcs look like polygons | The job came through the optimizer, or the axes have different scales. Read the causes in chapter 9. |
| A motor jumps | Type `diag`. Examine `step interrupt` and `pulses_sent`. Report the values with the job that caused the jump. |
| `error: disable motors first` | The setting needs the motors disabled. Type `disable`. |
| `error: a job is running` | The setting can change only between jobs. Wait until the job ends. |
| `applied but NVM write failed` | The supply voltage is too low to write the memory. Connect the 24 V supply and set the value again. |

APPENDIX A

# Console Commands

The console is on the USB port (type `menu` on a line by itself) and on TCP port 9300. Each command ends with a line that starts with `ok` or `error:`. When the console opens, it shows the firmware version, the model name, the IP address, the active links, and the source of the settings.

**Table A.1.** Console commands

| Command | Function |
|---|---|
| `help`, `menu` | Show the list of commands. |
| `status` | Show the state, the position in mm, the pen, the queue depth, the counters, and the data of the links (table 10.1). |
| `config` | Show all settings with their values, their descriptions, and when they take effect. |
| `set` *key value* | Make sure that the value is correct, apply it, and save it in the nonvolatile memory. Appendix B lists the keys. |
| `defaults` | Set all settings to the factory values. The motors must be disabled. |
| `enable` | Enable the motors. This also clears a latched fault. With `hlfb_check` 1, the board waits for HLFB from all servos. |
| `disable` | Disable the motors. A job that is in the queue is dropped. |
| `abort` | Stop with a ramp along the path (three times `accel_mm_s2`), drop the job, and raise the pen. This also stops a jog or homing. |
| `pause` | Stop at the end of the segments in the queue, and raise the pen. |
| `resume` | Lower the pen and continue after `pause`. |
| `origin` | The present position becomes plotter 0,0, and the step counters of the motors restart at 0. No job can be plotting. |
| `goto` *x_mm y_mm* | Move to the position with the pen up. No job can be plotting. The position stays in the plot area. |
| `pen up`, `pen down` | Raise or lower the pen to test it. No job can be plotting. |
| `clear` | Clear a latched fault. Remove the cause first. |
| `home` | Home each axis that has a home switch (`home_mode` 1), or both servos (`home_mode` 2). Section 6.5. |
| `home x`, `home y` | Home one axis. |
| `home stop` | Stop homing. |

| Command | Function |
|---|---|
| `jog` *axis mm_s* [`stopon` *inputs*] | Move an axis (`x` or `y`) at *mm_s* (negative for `-`). The jog stops 0.5 s after the last command, so send it again while it must continue. With `stopon DI-7,A-9` (for example), the jog stops when one of these inputs changes from its level at the start. Then the board refuses the jog until `jog stop`. A jog toward a tripped limit is refused. |
| `jog stop` | Stop the jog. |
| `swinfo` | Show the input levels, each switch (input, contact, state, limit stops, and where the last one tripped), the position, the jog, the last stop, and the last homing. |
| `sim` *input* `low\|high\|off` | Hold a simulated input low or high, or release it. |
| `sim` *input side mm* | A simulated switch on the input, *mm* from the head on *side* (`x+`, `x-`, `y+`, or `y-`). Section 6.7. |
| `sim clear` | Remove all simulated inputs. |
| `diag` | Show the motion diagnostics, the last job of the optimizer, and the data of the USB link. |
| `diag reset` | Set the timing counters to zero, then show the diagnostics. |
| `tel` *ms* | TCP console only. Send a line `TEL ms= x= y= pen= state= queue=` every *ms* milliseconds (10 or more; 0 stops it). A line is sent only when the TCP buffer has space, so a slow client cannot stop the motion. The desktop app uses 25 ms. |
| `restart` | Restart the board. |
| `version` | Show the name and the version of the firmware. |
| `exit`, `quit`, `q` | Close the console. On USB, the port goes back to HP-GL. |

## Error Messages

**Table A.2.** Common console errors

| Message | Meaning |
|---|---|
| `unknown key (try: config)` | The setting name is not correct. |
| `disable motors first` | The setting needs the motors disabled. |
| `a job is running; change it when the plotter is idle` | The setting can change only between jobs. |
| `applied but NVM write failed` | The board uses the value but could not save it. The supply voltage is too low. |
| `plotter busy` | The command needs the machine to be idle. |
| `that input already has a switch` | Two switches cannot use one input, except a home switch on a limit of the same axis. |
| `limit X+ tripped: jog the other way` | Move the axis away from the limit. |
| `stopped on a switch: DI-7 (jog stop)` | A `stopon` input stopped the jog. Type `jog stop`. |
| `unknown command (try: help)` | The command name is not correct. |

Appendix B

# Settings

The board keeps its settings in nonvolatile memory. Change a setting with `set` *key value* in the console, or in Plotter settings in the desktop app. `config` shows all values. `defaults` sets the factory values.

The column "When" tells you when a change is possible (table 5.1): **live** (at any time), **idle** (between jobs), **disabled** (with the motors disabled), or **restart** (after `restart`).

## B.1 Machine

**Table B.1.** Machine settings

| Key | Default | Range | When | Function |
|---|---|---|---|---|
| `kinematics` | 0 | 0 or 1 | disabled | 0: Cartesian (**M0** = X, **M1** = Y). 1: CoreXY or H-bot (**M0** = A, **M1** = B). Section 3.4. A change sets the origin. |
| `steps_rev_x` | 800 | 1–1,000,000 | disabled | **M0** steps per revolution. |
| `steps_rev_y` | 800 | 1–1,000,000 | disabled | **M1** steps per revolution. CoreXY: the same as **M0**. |
| `mm_rev_x` | 5.0 | 0.001–10,000 | disabled | Cartesian: X travel per motor revolution, mm. |
| `mm_rev_y` | 5.0 | 0.001–10,000 | disabled | Cartesian: Y travel per motor revolution, mm. |
| `pulley_dia_mm` | 12.7324 | 0.1–1000 | disabled | CoreXY: pulley pitch diameter, mm, both motors. Travel per revolution is $\pi$ × this value. |
| `dir_x` | 1 | 1 or -1 | disabled | Direction of X. -1 reverses it. |
| `dir_y` | 1 | 1 or -1 | disabled | Direction of Y. -1 reverses it. |
| `swap_xy` | 0 | 0 or 1 | disabled | 1: exchange the X and Y axes. A change sets the origin. |
| `width_mm` | 400 | 1–100,000 | idle | Plot area in X (the hard clip), mm. |
| `height_mm` | 300 | 1–100,000 | idle | Plot area in Y (the hard clip), mm. |

## B.2 Motion

**Table B.2.** Motion settings

| Key | Default | Range | When | Function |
|---|---|---|---|---|
| `draw_mm_s` | 100 | 0.1–10,000 | live | Pen-down speed, mm/s. `VS` without a value sets it back to this value. |
| `travel_mm_s` | 200 | 0.1–10,000 | live | Pen-up speed, mm/s. |
| `max_mm_s` | 250 | 0.1–10,000 | live | Highest speed that `VS` can set, mm/s. |
| `job_speed` | 1 | 0 or 1 | live | 1: `VS` in a job sets the pen-down speed. 0: always `draw_mm_s`. |
| `accel_mm_s2` | 2000 | 1–1,000,000 | live | Acceleration along the path, $mm/s^2$. Also limits the speed on curves. |
| `corner_mm_s` | 20 | 0–10,000 | live | Largest change of speed of one motor at a corner, mm/s. |
| `motor_max_mm_s` | 0 | 0–10,000 | live | Highest speed of a motor, as belt travel in mm/s. 0: no limit. Use it for CoreXY. |

## B.3 Pen

**Table B.3.** Pen settings

| Key | Default | Range | When | Function |
|---|---|---|---|---|
| `pen_mode` | 0 | 0 or 1 | disabled | 0: digital output. 1: servo on **M2**. |
| `pen_io` | 0 | 0–5 | disabled | Output for the pen, **IO-0**–**IO-5** (`pen_mode` 0). |
| `pen_active_high` | 1 | 0 or 1 | disabled | 1: the output is on when the pen is down. |
| `pen_down_ms` | 150 | 0–10,000 | live | Settle time after the pen goes down, ms. |
| `pen_up_ms` | 100 | 0–10,000 | live | Settle time after the pen goes up, ms. |
| `z_up_steps` | 0 | ±100,000,000 | disabled | **M2** pen-up position, steps. |
| `z_down_steps` | 800 | ±100,000,000 | disabled | **M2** pen-down position, steps. |
| `z_vel_steps` | 20,000 | 1–10,000,000 | disabled | **M2** speed, steps/s. |
| `z_accel_steps` | 400,000 | 1–100,000,000 | disabled | **M2** acceleration, $steps/s^2$. |

## B.4 Safety and Jobs

**Table B.4.** Safety and job settings

| Key | Default | Range | When | Function |
|---|---|---|---|---|
| `estop_di6` | 1 | 0–2 | live | 0: off. 1: fault when **DI-6** is low. 2: fault when it is high. |
| `auto_enable` | 1 | 0 or 1 | live | 1: enable the motors when a job arrives. |
| `hlfb_check` | 1 | 0 or 1 | live | 1: the servos must supply HLFB when the motors enable. |
| `optimize` | 3 | 0–7 | live | Sources that the optimizer orders: 1 USB, 2 COM, 4 network. Add the values. |
| `ident` | `7475A` | text | live | Reply to `OI`. `758x` and `759x` (DraftMaster) put plotter 0,0 at the center of the bed. |

## B.5 Limits and Homing

Chapter 6 tells how to use these settings. An input is `none`, `DI-7`, `DI-8`, `A-9` to `A-12`, or `IO-0` to `IO-5`. The `x_home_*` settings have `y_home_*` settings for Y with the same defaults.

**Table B.5.** Limit and homing settings

| Key | Default | Range | When | Function |
|---|---|---|---|---|
| `x_limit_plus` | none | input | idle | X+ limit switch. |
| `x_limit_minus` | none | input | idle | X- limit switch. |
| `x_home` | none | input | idle | X home switch. It can be the input of an X limit. |
| `y_limit_plus` | none | input | idle | Y+ limit switch. |
| `y_limit_minus` | none | input | idle | Y- limit switch. |
| `y_home` | none | input | idle | Y home switch. It can be the input of a Y limit. |
| `switch_open` | 0 | 0–63 | idle | Normally open switches: 1 X+, 2 X-, 4 X home, 8 Y+, 16 Y-, 32 Y home. Add the values. |
| `home_mode` | 0 | 0–2 | idle | 0: no homing. 1: home on the home switches. 2: ClearPath hard-stop homing (Cartesian). |
| `home_required` | 0 | 0 or 1 | live | 1: home before a job, if the axes are not homed. |
| `home_order` | 0 | 0 or 1 | live | 0: X, then Y. 1: Y, then X. |
| `x_home_dir` | -1 | -1 or 1 | idle | Direction toward the X home switch. |

| Key | Default | Range | When | Function |
|---|---|---|---|---|
| x_home_fast | 20 | 0.1–1000 | idle | Search speed, mm/s. |
| x_home_slow | 2 | 0.05–100 | idle | Latch speed, mm/s. |
| x_home_back | 3 | 0.1–100 | idle | Back-off after the switch releases, mm. |
| x_home_at | 0 | ±100,000 | idle | X position at the switch, mm. |
| x_home_travel | 0 | 0–100,000 | idle | Longest search, mm. 0: 1.2 times the plot area plus 20 mm. |

## B.6 Serial Port and Network

**Table B.6.** Serial port and network settings

| Key | Default | Range | When | Function |
|---|---|---|---|---|
| com_port | 0 | 0–2 | restart | HP-GL serial port. 0: off. 1: **COM-0**. 2: **COM-1**. |
| com_baud | 9600 | 300–1,000,000 | restart | Baud rate of the serial port. |
| com_handshake | 1 | 0–2 | restart | 0: none. 1: XON/XOFF. 2: RTS/CTS. |
| com_rs232 | 1 | 0 or 1 | restart | 1: RS-232 levels. 0: TTL levels. |
| net_mode | 0 | 0 or 1 | restart | 0: DHCP. 1: static address. |
| ip | – | address | restart | Static IP address. |
| netmask | – | address | restart | Static netmask. |
| gateway | – | address | restart | Static gateway. |

> **NOTE:** With DHCP, if the board gets no address, it uses 192.168.0.110.

Appendix C

# HP-GL Support

The board operates as an HP-GL pen plotter of the HP 7475A class. It also reads the HP-GL/2 instructions that drivers usually send. The HP-GL/2 reference guide gives the full definition of each instruction.[1]

## C.1 Syntax

- Mnemonics are not case sensitive.
- Commas or spaces separate parameters.
- An instruction ends with `;` or with the next mnemonic, for example `PA10,10PD20,20`.
- Decimal values and signs are correct in all parameters.

## C.2 Instructions

**Table C.1.** HP-GL instructions

| Group | Instructions | Notes |
|---|---|---|
| Setup | `IN` `DF` `IP` `IR` `SC` (types 0, 1, and 2) `IW` `RO` | `IN` raises the pen, sets P1 and P2 to the plot area, and clears errors. |
| Pen and motion | `PU` `PD` `PA` `PR` `PE` | `PE` is the HP-GL/2 encoded polyline, with all its flags (base 64, 7-bit, fractional bits, absolute, pen up, and pen select). |
| Pen and speed | `SP` `VS` | One pen. `SP0` raises the pen. `VS` is in cm/s, up to `max_mm_s`. |
| Arcs and shapes | `AA` `AR` `CI` `CT` `EA` `ER` `EW` `RA` `RR` `WG` `XT` `YT` `TL` | Pen-down arcs are true arcs. `CI` and `EW` go back to the center. `RA`, `RR`, and `WG` draw only the outline. |
| Page | `PG` `NR` `AF` `AH` `FR` | Raise the pen and count a page. |
| Output | `OI` `OS` `OE` `OA` `OC` `OP` `OH` `OW` `OF` `OO` `OT` `OD` `OG` `OL` | `OI` replies with `ident`. `OS` clears bits 8 and 2 after it is read. `OE` clears the error. |
| Text | `LB` `DT` | Not drawn. The board reads the text up to the terminator and counts it. |
| No effect | `LT` `PT` `FT` `LA` `PW` `WU` `NP` `PC` `BP` `PS` `QL` `RP` `MC` `CO` `MG` `SI` `SR` `DI` `DR` `CS` `CA` `SS` `SA` `ES` `LO` `CP` `SM`, and others | Accepted and ignored. All lines are solid. Strings in quotes are skipped. |

[1] Hewlett-Packard Company, *The HP-GL/2 and HP RTL Reference Guide: A Handbook for Program Developers*, 3rd ed. (Reading, MA: Addison-Wesley, 1997).

An instruction that is not in table C.1 sets error 1 and is skipped. The board removes `@PJL` lines and the PCL and PJL escape sequences that Windows drivers send.

## C.3 Errors and Status

**Table C.2.** Error codes (`OE`)

| Code | Meaning |
|---|---|
| 0 | No error. |
| 1 | Instruction not known. |
| 2 | Wrong number of parameters. |
| 3 | Parameter not correct (for example `R045`, or `SC` with a range of zero). |
| 6 | Position overflow. |

**Table C.3.** Status bits (`OS`)

| Bit | Value | Meaning |
|---|---|---|
| 0 | 1 | Pen down. |
| 1 | 2 | P1 or P2 changed (cleared by `OS` or `OP`). |
| 3 | 8 | Initialized (set by `IN`, cleared by `OS`). |
| 4 | 16 | Ready. |
| 5 | 32 | Error (cleared by `OE`). |

At power-up, `OS` replies 24.

## C.4 Device Control (ESC .)

The board answers these sequences immediately, before the data in its buffer, as an HP 7475A does. They operate on all links. The reply ends with a carriage return; `ESC.M` can change this.

**Table C.4.** Device-control sequences

| Sequence | Function or reply |
|---|---|
| `ESC.( ESC.Y` | Plotter on. |
| `ESC.) ESC.Z` | Plotter off. The board discards data until the plotter is on. |
| `ESC.B` | Free bytes in the buffer. |
| `ESC.L` | Size of the buffer. |
| `ESC.O` | 8 if the buffer is empty, or 0. |
| `ESC.E` | Extended error, then clear it: 11 bad byte after `ESC.`, 12 bad parameter, 14 too many parameters, 16 buffer overflow. |
| `ESC.K` | Stop the graphics: drop the buffered data, stop the motion, and raise the pen. |

| Sequence | Function or reply |
|---|---|
| ESC.J | Stop device control (cancel an ACK that waits). |
| ESC.R | Reset the handshake and the output settings. |
| ESC.H, ESC.I | ENQ/ACK handshake. |
| ESC.N | Enable XON/XOFF with the given XOFF character. |
| ESC.M | Set the output terminator. |
| ESC.@ ESC.S ESC.T ESC.U ESC.P | Accepted, no effect. |

## C.5 Flow Control

**Table C.5.** Flow control on each link

| Link | Mechanism |
|---|---|
| TCP 9100 | The board stops reading when its 8 KB buffer is full. |
| USB | The board stops reading when its 4 KB buffer is full. |
| COM | com_handshake: 1 XON/XOFF (XOFF at less than 128 bytes free, XON above half), 2 RTS/CTS, 0 none. The buffer is 1024 bytes. ENQ/ACK and ESC.B operate on all links. |

Appendix D

# Network and Command-Line Client

## D.1 Ports

**Table D.1.** Network ports of the board

| Port | Function |
|---|---|
| TCP 9100 | HP-GL, raw print port. One connection is one job. |
| TCP 9300 | Setup and status console. One client at a time. |
| UDP 9302 | Discovery. |

These ports are different from the ports of ClearCNC (8888, 8889, 10040) and ClearCoreROS (9200–9203).

## D.2 Discovery

To find boards, a computer sends the text `CLEARHPGL_DISCOVER?` as a UDP broadcast to port 9302. Each board replies with one line:

```
CLEARHPGL ClearHPGL_Firmware IP=10.168.41.5 HPGL=9100 CONSOLE=9300 FW=0.2.0 IDENT=7475A
```

The Discover button of the desktop app and the `discover` command of the client use this method. A broadcast does not go through a router, so discovery finds only boards on the local subnet.

## D.3 The Command-Line Client

`Firmware/host/clearhpgl_cli.py` is a client for the board. It needs Python 3. For COM ports, it also needs the `pyserial` package.

**Table D.2.** Commands of the client

| Command | Function |
|---|---|
| `discover` | Find boards on the local network. |
| `--host` *ip* `send` *file* | Send an HP-GL file to TCP port 9100. |
| `--serial COM3 send` *file* | Send an HP-GL file to a COM port with XON/XOFF, as a plotter driver does. |
| `--host` *ip* `query "OI;OS;"` | Send output instructions and show the replies. |
| `--host` *ip* `console` *command* | Send one console command, for example `console status`. |

Examples:

```
python Firmware/host/clearhpgl_cli.py discover
python Firmware/host/clearhpgl_cli.py --host 10.168.41.5 send drawing.plt
python Firmware/host/clearhpgl_cli.py --host 10.168.41.5 query "OI;OS;OA;"
python Firmware/host/clearhpgl_cli.py --host 10.168.41.5 console set draw_mm_s 80
```

> **NOTE:** The `console` command connects to port 9300. If the desktop app is connected over Ethernet, it has the console, and the client cannot connect. Close the app connection first.

## D.4 Test without a Board

`Firmware/host/fake_plotter.py` operates as a board on the computer. Use it to try the desktop app without hardware:

```
python Firmware/host/fake_plotter.py --bind 127.0.0.1
```

Then connect the app to 127.0.0.1.

# Glossary

**Arc, true**
An arc that the motion planner draws as one smooth movement on a circle, not as short straight lines.

**Chord**
A short straight line that replaces a part of an arc.

**ClearCore**
The I/O and motion controller from Teknic that runs the ClearHPGL firmware.

**ClearPath**
A servo motor from Teknic with an integrated drive. The ClearPath-SD model receives step and direction signals.

**Clip window**
A rectangle, set with `IW`, outside which the board does not draw.

**CoreXY, H-bot**
Machines in which two belts move the head together. Motor A moves by X + Y and motor B by X − Y. Set with `kinematics` 1.

**Console**
The text interface of the board for setup and diagnosis, on USB and on TCP port 9300.

**DHCP**
A network service that gives the board its IP address automatically.

**DXF**
A file format for drawings from AutoCAD and other CAD software.

**Hard clip**
The plot area: the rectangle `width_mm` by `height_mm` from plotter 0,0. The board does not draw outside it.

**HLFB**
High-Level Feedback. A signal from a ClearPath servo that tells the controller that the servo is ready.

**HP-GL, HP-GL/2**
The plotter languages of HP. HP-GL/2 adds encoded polylines and other instructions.

**Job**
All the data that one source sends for one plot.

**Job optimizer**
The part of the firmware that keeps a full job and plots it in a better order.

**Nonvolatile memory (NVM)**
Memory in the ClearCore that keeps the settings when the power is off.

**Origin**
Plotter 0,0. The position of the pen at power-up, or the position where you used `origin`.

**Plot area**
See *hard clip*.

**Plotter unit**
The unit of HP-GL coordinates: 0.025 mm.

**Settle time**
The time that the board waits after the pen goes down or up, before the axes move.

**Stroke**
A pen-down line or a chain of pen-down lines without a pen-up move.

**STE**
ASD-STE100 Simplified Technical English, the controlled language of this manual.

# Bibliography


ASD. *ASD-STE100: Simplified Technical English*. Issue 8. Brussels: ASD, 2021.

*The Chicago Manual of Style*. 18th ed. Chicago: University of Chicago Press, 2024.

Hewlett-Packard Company. *The HP-GL/2 and HP RTL Reference Guide: A Handbook for Program Developers*. 3rd ed. Reading, MA: Addison-Wesley, 1997.

Sweeney, Adam G. "ClearHPGL." Source code repository, firmware version 0.2.0, 2026. Includes `Firmware/PROTOCOL.md`.

Sweeney, Adam G. "EnhancedClearCoreLibrary." Source code repository, 2026. `https://github.com/AGSweeney/EnhancedClearCoreLibrary`.

Teknic, Inc. *ClearCore Manual*. Victor, NY: Teknic. `https://teknic.com/products/io-motion-controller/`.

Teknic, Inc. *ClearPath User Manual*. Victor, NY: Teknic.